

The Theory and Practice of MAP Inference over Non-Convex Constraints

Leander Kurscheidt^{*1} Gabriele Masina^{*2} Roberto Sebastiani² Antonio Vergari¹

Abstract

In many safety-critical settings, probabilistic ML systems have to make predictions subject to algebraic constraints, e.g., predicting the most likely trajectory that does not cross obstacles. These real-world constraints are rarely convex, nor are the densities considered (log-)concave. This makes computing this constrained maximum a posteriori (MAP) prediction efficiently and reliably extremely challenging. In this paper, we first investigate under which conditions we can perform constrained MAP inference over continuous variables exactly and efficiently, and devise a scalable message-passing algorithm for this tractable fragment. Then, we devise a general constrained MAP strategy that interleaves partitioning the domain into convex feasible regions with numerical constrained optimization. We evaluate both methods on synthetic and real-world benchmarks, showing our approaches outperform constraint-agnostic baselines, and scale to complex densities intractable for SoTA exact solvers.

1. Intro

Making predictions with probabilistic machine learning (ML) models can be mapped to performing *maximum a posteriori* (MAP; Bishop 2006) inference, i.e., computing the output configuration with the highest likelihood according to the distribution learned by the model. However, in several real-world scenarios, from physics applications (Hansen et al., 2023; Cheng et al., 2024) to fair predictions (Ghandi et al., 2024) and “what-if” time-series analysis (Narasimhan et al., 2024), distributions are *constrained*, i.e., some configurations are infeasible and should never be predicted nor sampled (Grivas et al., 2024). This is mandatory if ML models are deployed in safety-critical scenarios (Giunchiglia

^{*}Equal contribution ¹School of Informatics, University of Edinburgh, UK ²DISI, University of Trento, Italy. Correspondence to: Leander Kurscheidt <l.kurscheidt@sms.ed.ac.uk>, Gabriele Masina <gabriele.masina@unitn.it>.

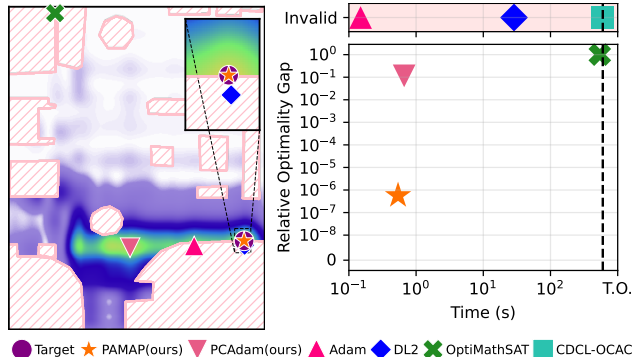


Figure 1. Our structure-aware solver PAMAP is able to correctly and efficiently perform MAP inference over non-convex constraints and non-log-concave densities, while classical optimizers (Adam), even when being constraint-aware (PCADAM, DL2; Sec. 6), are imprecise and slower, and exact solvers (OptiMathSAT (Sebastiani & Trentin, 2020) and CDCL-OCAC (Jia et al., 2025)) timeout. If the density factorizes as a tree, our MPMAP solver (Sec. 4) can be exact and even faster (see Fig. 6).

et al., 2023; Bortolotti et al., 2024).

Constrained MAP inference is well understood from an optimization perspective when it comes to discrete variables (Marinescu & Dechter, 2004; Martins et al., 2011) or when the problem has a simple form, i.e., constraints are convex (Dantzig, 2002; Jaggi, 2013) and distributions are log-concave (Doss & Wellner, 2019), as we discuss in Sec. 3. However, these assumptions are generally not met in real-world applications (De Smet et al., 2023; Kurscheidt et al., 2025; Stoian & Giunchiglia, 2025) and understanding how to perform efficient MAP inference over non-convex constraints and non-log-concave distributions is an open and challenging problem. In this paper, we reduce this gap by advancing a number of contributions, discussed next.

First, **C1**) we theoretically trace a non-trivial fragment of tractable constrained MAP problems over non-convex constraints and distributions represented as tree-factorized piecewise (exponentiated) polynomials. We then prove that it can be solved by an efficient message-passing scheme (MPMAP; Sec. 4). Our MPMAP is inspired by message-passing schemes to compute probabilities under non-convex constraints (Zeng et al., 2020a;b), but differently from them, performing MAP inference yields different challenges and different complexity results.

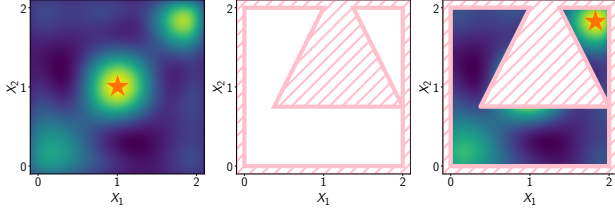


Figure 2. An example of $\text{MAP}(\mathcal{LR}\mathcal{A})$ inference over non-convex constraints and non-log-concave density. Left: an unconstrained density in 2D. Center: non-convex constraints. Right: constrained density. Orange stars indicate the solutions for the MAP problem for the unconstrained and constrained densities.

Second, **C2**) we investigate how to approximate constrained MAP via optimization for general constraints and distributions beyond the tree-factorized setting. To this end, we design PAMAP, a general scheme that decomposes global optimization into a series of MAP inference problems over convex constraints that can be efficiently solved by standard optimizers, and can provide approximation guarantees for polynomial densities (Powers & Wörmann, 1998; Lasserre, 2001). Lastly, **C3**) we rigorously evaluate MPMAP and PAMAP over a set of synthetic and real-world benchmarks, reporting that our custom optimizers are able to outperform a number of SoTA optimizers (Sebastiani & Trentin, 2020; Jia et al., 2025) both in terms of approximation quality and time to achieve it (see Fig. 1). As such, we set the first milestone to tackle the challenging problem of constrained optimization from both theory and practice.

2. Maximum A Posteriori Inference Under Non-Convex Algebraic Constraints

Notation. We denote random variables by uppercase letters (e.g., X, Y), and their assignments by lowercase ones (e.g., x, y). Bold symbols denote sets of variables (e.g., $\mathbf{X}, \mathbf{Y}$), and their joint assignments (e.g., $\mathbf{x}, \mathbf{y}$). Greek letters such as Δ denote logical formulas that map real values to binary values (false, true). We say that assignment $\mathbf{x}$ satisfies the constraint Δ , denoted as $\mathbf{x} \models \Delta$, if substituting $\mathbf{x}$ into Δ makes Δ true. So, the indicator function $\llbracket \mathbf{x} \models \Delta \rrbracket$ is 1 when $\mathbf{x}$ satisfies Δ , and 0 otherwise.

SMT($\mathcal{LR}\mathcal{A}$) Constraints. We consider algebraic constraints over $\mathbf{X}$ representing a collection of non-convex polytopes.¹ We express them in the language of *satisfiability modulo theory over linear real arithmetic* (SMT($\mathcal{LR}\mathcal{A}$); Barrett et al. 2021), hereafter just SMT for short. We consider quantifier-free SMT-formulas over continuous variables $\mathbf{X}$, where linear (in)equalities $\sum_i a_i X_i \bowtie b$, with

¹In this paper, the term *polytope* refers to a *convex polytope*, while *non-convex polytope* refers to a more general geometric object defined as a connected set with flat faces (as, e.g., in Chin et al. 2015).

$\bowtie \in \{\leq, =\}$, are connected via Boolean connectives —i.e., conjunctions ($\wedge$), disjunctions ($\vee$), and negations ($\neg$). As we show in Fig. 1, SMT is flexible enough to represent rich real-world constraints. We now provide a simpler example.

Example 2.1 (SMT formula). Consider the following SMT formula over variables $\mathbf{X} = \{X_1, X_2\}$:

$$\Delta(\mathbf{X}) = X_1 \in [0, 2] \wedge X_2 \in [0, 2] \wedge ((X_2 \leq 1) \vee (X_2 > 2X_1) \vee (X_2 > 4.75 - 2X_1)).$$

Δ defines a square feasible region containing an inner infeasible triangle, as shown in Fig. 2 (center).

MAP under SMT constraints. Given a probability density function $p(\mathbf{X})$ over continuous random variables $\mathbf{X} = \{X_1, \dots, X_n\}$ and some SMT constraints Δ over $\mathbf{X}$, the goal of constrained MAP inference is to find the assignment $\mathbf{x}^*$ that maximizes $p(\mathbf{X})$ while satisfying Δ :

$$\operatorname{argmax}_{\mathbf{x} \models \Delta} p(\mathbf{x}) = \operatorname{argmax}_{\mathbf{x}} \tilde{p}(\mathbf{x}) \llbracket \mathbf{x} \models \Delta \rrbracket \quad (\text{MAP}(\mathcal{LR}\mathcal{A}))$$

where $\tilde{p}$ can be an unnormalized density, i.e., a nonnegative function.² Clearly, the solution to $\text{MAP}(\mathcal{LR}\mathcal{A})$ can drastically differ from the one for MAP inference over an unconstrained distribution.

Example 2.2 (Constrained MAP). Consider the SMT formula of Ex. 2.1 and the unnormalized polynomial shown in Fig. 2 (left). The unconstrained optimum is (1.0, 1.0), but the constrained one is (1.83, 1.83), shown on the right. Note also that the new optimum is in a disconnected polytope, highlighting how the optimization problem for $\text{MAP}(\mathcal{LR}\mathcal{A})$ has a combinatorial nature that makes it challenging for traditional continuous optimizers.

Examples of $\text{MAP}(\mathcal{LR}\mathcal{A})$ problems can be found in several recent works where SMT constraints are defined over the output of a neural network (De Smet et al., 2023; Kurscheidt et al., 2025). Since in those scenarios we deal with conditional distributions, i.e., we want to solve $\operatorname{argmax}_{\mathbf{y} \models \Delta} p(\mathbf{y} \mid \mathbf{x})$, we have to solve one $\text{MAP}(\mathcal{LR}\mathcal{A})$ problem for each datapoint $\mathbf{x}$. This motivates us to find fast and reliable solvers that can be parallelized. Next, we review established methods for solving $\text{MAP}(\mathcal{LR}\mathcal{A})$ when the distribution p and constraints Δ take simpler forms.

3. The SoTA of Constrained MAP Inference

Solving $\text{MAP}(\mathcal{LR}\mathcal{A})$ exactly is generally computationally hard, as even optimizing a quadratic function over a convex polytope is NP-hard (Sahni, 1974). However, efficient methods for specific problem classes exist, each one trading off

²If Δ does not define a closed set, the maximum may not be attained; in this case we seek the best finite-precision approximation of the $\operatorname{arg sup}$.

expressiveness, efficiency, and optimality guarantees in a different way.

Convex constraints, log-concave densities. If the constraint Δ is *convex*, e.g., it is defined as a *conjunction* of linear constraints, and the objective is (*log*-) *concave*, any local optimum is also a global optimum. This property ensures that algorithms such as sequential quadratic programming (Boggs & Tolle, 1995) and interior-point methods (Conn et al., 2000) are guaranteed to efficiently converge to the global optimum.

Convex constraints and piecewise densities. When the density decomposes into several pieces of simple form, one can go beyond log-concavity and use symbolic bucket elimination for *constrained piecewise functions* as in Ye et al. (2018). The latter tackles the problem of optimizing a sum of piecewise linear or univariate quadratic (LUQF) functions, i.e., only one variable can be quadratic in each piece, under convex linear constraints. The approach is based on the XADD data structure (Sanner & McAllester, 2005) and leverages the symbolic, partial maximization algorithm of Zamani et al. (2021). A similar line of work by Jeong et al. (2023) deals with mixed-integer linear programming over convex, and linear constraints which decompose into two sets of constraints over disjoint sets of variables.

General polynomial densities. For non-convex problems, exact global constrained optimization is notoriously challenging, even if p is a polynomial. For instance, the *cylindrical algebraic decomposition* (CAD; Collins 1975; Arnon et al. 1984; Wolfram Research) has doubly-exponential worst-case complexity in the number of variables. A classical way to approximate $\text{MAP}(\mathcal{LR}, \mathcal{A})$ with guarantees for polynomial densities is the Lasserre hierarchy (Lasserre, 2001), which solves a sequence of semidefinite programming (SDP) relaxations. As the relaxation order increases, the solution provides increasingly tight upper bounds, converging to the global optimum in the limit. Under linear constraints, it is possible to compute a lower bounds on the global maximum. The dual formulation of the problem can be solved via *sum-of-squares* optimization (SoS), which also provides a sequence of improving upper bounds converging to the global maximum.

Local optimizers for general densities and convex constraints. A practical way to approximate $\text{MAP}(\mathcal{LR}, \mathcal{A})$ when Δ is convex, is to run a local optimizer starting from multiple initial points (particles), so to increase the chances of finding the global optimum. E.g., Basin Hopping (Wales & Doye, 1997) combines local minimization with stochastic perturbations of the current solution, whereas SHGO (Endres et al., 2018) exploits a simplicial complex to identify locally convex subdomains corresponding to distinct minima, so to systematically explore the objective landscape. These methods scale well with problem dimensionality and make

minimal assumptions about the shape of the constraints or objective function—typically requiring only smoothness or Lipschitz continuity—but provide no guarantees of finding the global optimum in the general case. We leverage these optimizers in our PAMAP in Sec. 5 when we decompose a non-convex constraint into convex polytopes.

Optimization under general SMT constraints. Optimization under SMT constraints is known in the literature as *optimization modulo theories* (OMT; Nieuwenhuis & Oliv-eras 2006; Sebastiani & Tomasi 2012; 2015). In OMT, however, both the objective function (our density) and the constraint need to belong to the same family. For linear constraints and objectives, OMT can be solved efficiently using an SMT solver extended with optimization capabilities (Sebastiani & Tomasi, 2012; Li et al., 2014; Bjørner et al., 2015). The extension to non-linear polynomial constraints and objective functions, however, is non-trivial and remains an active area of research. OptiMathSAT (Sebastiani & Trentin, 2020) uses an incomplete approach, solving a linear overapproximation of the original non-linear problem which is iteratively refined (Bigarella et al., 2021). Recently, Jia et al. (2025) proposed CDCL-OCAC, a complete algorithm based on a variant of CAD, hence suffering from the same scalability limitations. Finally, we remark that the extension of common optimization methods that guarantee feasibility, such as barrier methods (Nocedal & Wright, 2006), to SMT constraints with a complex Boolean structure is not straightforward. Indeed, to the best of our knowledge, no such extension exists in the literature, making these methods not directly applicable to $\text{MAP}(\mathcal{LR}, \mathcal{A})$.

Soft-constrained optimization. Rather than guaranteeing feasibility, soft-constrained optimization only encourages constraint satisfaction by adding a penalty term to the objective (Xu et al., 2017; Smet et al., 2023; Fischer et al., 2019). As the indicator function over the constraints is not differentiable and therefore unsuitable for gradient-based optimization, various differentiable relaxations have been proposed. Xu et al. (2017) introduces a semantic loss for propositional logic; Stewart & Ermon (2016) encodes physical laws into penalties; and Diligenti et al. (2017) leverages fuzzy logic to translate logical statements into differentiable objectives. For $\text{SMT}(\mathcal{LR}, \mathcal{A})$, DL2 (Fischer et al., 2019) provides a framework to translate algebraic constraints into penalties suitable for optimization.

4. A Scalable Message-Passing Algorithm For Constrained MAP

As discussed in the previous section, tractable fragments of $\text{MAP}(\mathcal{LR}, \mathcal{A})$ are restricted to convex constraints and piecewise linear or univariate quadratic functions (Ye et al., 2018). As a first contribution **C1**, we now relax these requirements to cover a larger class of tractable constrained MAP prob-

lems, involving non (log-)concave densities and constraints factorizing as a tree. We do so by building on prior work that enables exact integration over algebraic constraints, also known as weighted model integration (WMI) (Belle et al., 2015; Morettin et al., 2017), via a message-passing scheme (Zeng et al., 2020a;b). Closely related is prior work on integrating Gaussian probabilities over polyhedral constraints using message passing (Cunningham et al., 2011). While our approach takes inspiration from WMI, we show that the class of tractable densities for WMI is not suitable for tractable MAP($\mathcal{LR}\mathcal{A}$).

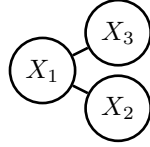
A tree-shaped MAP($\mathcal{LR}\mathcal{A}$). Following Zeng et al. (2020a;b), we consider problems where the product between the density and constraints $\llbracket \mathbf{x} \models \Delta \rrbracket \cdot p(\mathbf{x})$ decomposes with a tree-shaped graph structure. We begin with the SMT formula Δ , which we assume is in conjunctive normal form (CNF) and with at most bivariate clauses. As a consequence, the indicator function factorizes as

$$\llbracket \mathbf{x} \models \Delta \rrbracket = \prod_{X_i, X_j \in \mathcal{E}_\Delta} \llbracket (x_i, x_j) \models \Delta_{ij} \rrbracket \prod_{v \in V_\Delta} \llbracket x_v \models \Delta_v \rrbracket \quad (1)$$

where $\mathcal{E}_\Delta$ (resp. V_Δ) is the set of pairs of variables appearing in the same bivariate clause (resp. univariate clauses), and Δ_S is the restriction of Δ to the clauses over the variables in S . Furthermore, we require the graph $(\mathbf{X}, \mathcal{E}_\Delta)$, also called the *primal graph* of Δ , to have a treewidth of one, thus encoding a tree (or a forest), as shown next.

Example 4.1 (Primal graph of SMT formula). The following SMT formula over variables $\mathbf{X} = \{X_1, X_2, X_3\}$ (left) exhibits a tree-shaped primal graph $(\mathbf{X}, \mathcal{E}_\Delta)$ (right):

$$\begin{aligned} \Delta_i &= |X_i| \leq 1 \\ \Delta_{1i} &= 1 \leq |X_1 - X_i| \leq 2 \\ \Delta &= \Delta_{12} \wedge \Delta_{13} \wedge \bigwedge_{i=1}^3 \Delta_i \end{aligned}$$



Additionally, a similar structure is assumed for p , which we also assume to factorize into at most bivariate components:

$$p(\mathbf{x}) = \prod_{X_i, X_j \in \mathcal{E}_p} p_{ij}(x_i, x_j) \prod_{X_i \in \mathbf{X}} p_i(x_i) \quad (2)$$

Here, the set $\mathcal{E}_p$ denotes the set of variable pairs appearing in the domains of the bivariate functions p_{ij} . We also assume the graph $(\mathbf{X}, \mathcal{E}_p)$ to have a treewidth of one. This is not enough, however. In order to tractably compute MAP($\mathcal{LR}\mathcal{A}$), we need a few tractable operations and properties over the functions p_{ij} and p_i :

Definition 4.2 (Tractable MAP Conditions). We say that the *tractable MAP conditions* (TMC) hold for a family of functions Ω if we have:

- (i) **Closedness under product:** $\forall f, g \in \Omega : f \cdot g \in \Omega$;
- (ii) **Tractable symbolic supremum:** For any bivariate $f \in \Omega$ and linear bounds $l(x_i), u(x_i)$, both $m(x_i) :=$

$\sup_{x_j \in [l(x_i), u(x_i)]} f(x_i, x_j)$ and its arg sup belong to Ω and can be computed tractably;

- (iii) **Tractable pointwise maximum:** For any univariate $f, g \in \Omega$, $o(x) := (\arg) \max\{f(x), g(x)\}$ belongs to Ω and can be computed tractably.

We note that the function family identified by Zeng et al. (2020a) that enables tractable integration over SMT formulas (tractable WMI) does not necessarily satisfy our TMC conditions. For example, general piecewise polynomials enable tractable WMI, but can violate property (ii) of Def. 4.2, as we discuss in Appendix A.1. To identify a function class satisfying Def. 4.2, we have to add further properties to polynomials. To this end, we identify two major families of functions for which TMC holds: Ω^{PP} and Ω^{PEP} . Ω^{PP} is the family of piecewise polynomial functions whose domain is partitioned by conjunctions of linear inequalities, and each polynomial *factorizes* into a product of two univariate polynomials.

Example 4.3 (Example of a density in Ω^{PP}). The density p over $\mathbf{X} = \{X_1, X_2, X_3\} \in [-1, 1]^3$ that factorizes as

$$\begin{aligned} p_1(x_1) &= 0.05 & p_2(x_2) &= (x_2 + 1) \\ p_{1,3}(x_1, x_3) &= (1 - x_1)(3 - x_3) & p_3(x_2) &= (1 - x_3) \\ p_{1,2}(x_1, x_2) &= 0.2 \cdot (x_1 - 0.9)^2 (x_2 + 0.9)^2 \llbracket x_1 - x_2 < 0 \rrbracket \\ p_{1,2}(x_1, x_2) &= (x_1 + 1) \llbracket x_1 - x_2 \geq 0 \wedge x_1 \leq 0.5 \rrbracket \end{aligned}$$

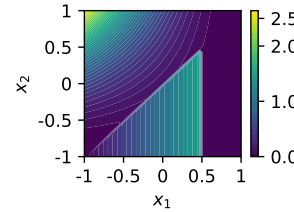


Figure 3. Densities in Ω^{PP} can express complex and multimodal densities as shown here for the density $p_{1,2}(x_1, x_2)$ from Ex. 4.3. Despite factorizing into univariate polynomials, different pieces can recover correlations.

Similarly, Ω^{PEP} is the family of piecewise exponentiated polynomials, where each piece factorizes into a product of exponentiated univariate polynomials. A prominent example of Ω^{PEP} is the multivariate Gaussian with independent components. As such, our TMC conditions cover functions that do not support tractable integration according to Zeng et al. (2020a), hinting to the fact that WMI and MAP($\mathcal{LR}\mathcal{A}$) are incomparable problems (see Appendix A.1).

To properly define a tractable fragment of MAP($\mathcal{LR}\mathcal{A}$), we need to enforce the global problem structure, defined next, to be a tree. That, in turn, leads to our first main result.

Definition 4.4 (Global graph). The *global graph* of a MAP($\mathcal{LR}\mathcal{A}$) problem over an SMT formula Δ and density p factorizing as (1) and (2) is the graph $\mathcal{G} = (\mathbf{X}, \mathcal{E})$ with $\mathcal{E} := \mathcal{E}_\Delta \cup \mathcal{E}_p$.

Theorem 4.5 (Tractability of MAP($\mathcal{LR}\mathcal{A}$)). *If the global graph of MAP($\mathcal{LR}\mathcal{A}$) has treewidth one and bounded di-*

ameter, and the density fulfills the TMC (Def. 4.2), then MAP($\mathcal{LR.A}$) can be solved tractably.

The proof is by construction and detailed in Appendix A.2. It proceeds by building a fixed-parameter tractable message-passing algorithm, which we discuss next.

As a final note, we point out that even though the tree-shaped structure is a strong requirement, it is not uncommon. Tree-shaped dependencies arise naturally in many fields, such as phylogenetic trees, hierarchical file and network systems, and natural language grammars (Zeng et al., 2020a). Furthermore, trees are frequently used as effective approximations for more complex, non-tree-shaped problems (Zeng et al., 2020c). Understanding how far we can push tractable computation over trees is thus a crucial step towards understanding the limits of tractable MAP($\mathcal{LR.A}$).

MPMAP. The key idea behind our message-passing algorithm for constrained MAP (MPMAP) is to exploit the tree structure of the global graph. By iteratively conditioning on a variable node, we render its children independent, and allow maximization to be performed independently over each sub-tree. We can therefore decompose the computation of the overall maximum into smaller problems until we arrive at the leaves. We start by reordering $\llbracket \mathbf{x} \models \Delta \rrbracket \cdot p(\mathbf{x})$ by grouping them via $\mathcal{E}$ and introduce the factor representation:

$$F_S(\mathbf{x}_S) := \llbracket \mathbf{x}_S \models \Delta_S \rrbracket \cdot p_S(\mathbf{x}_S) \quad (3)$$

with the scope $\mathcal{S}$ being over both variables and edges in $\mathcal{E}$ (e.g. F_{13}) and single variable indices in $\mathbf{X}$ (e.g. F_2). We set Δ_S to True and p_S to 1 if not previously defined in Δ and p . This results in $\llbracket \mathbf{x} \models \Delta \rrbracket \cdot p(\mathbf{x}) = \prod_S F_S(\mathbf{x}_S)$.

If we manage to “maximize out” (akin to marginalizing out) the variables one by one —e.g., in the example calculate $\max_{x_i} F_{i1}(x_i, x_1) F_i(x_i)$ as a function of x_1 — we have a scalable algorithm to compute exactly the constrained MAP even for high-dimensional problems. This can be extended to the argmax by not only tracking the value but also the position at which the value is attained, and then backtrack, as detailed in the Appendix A.3. We start from a root node X_r , that can be chosen so to minimize the number of computations, and then recurse into the children of the directed global graph until we hit the leaves:

$$m_{X_i \rightarrow F_{ij}}(x_i) := \prod_{c \in \text{child}(X_i)} m_{F_{ci} \rightarrow X_i}(x_i) \cdot F_i(x_i) \quad (4)$$

$$m_{F_{ij} \rightarrow X_j}(x_j) := \max_{x_i} F_{ij}(x_i, x_j) \cdot m_{X_i \rightarrow F_{ij}}(x_i) \quad (5)$$

$$\max_{\mathbf{x}} \prod_S F_S(\mathbf{x}_S) = \max_{x_r} F_r(x_r) \cdot \prod_{c \in \text{child}(X_r)} m_{F_{cr} \rightarrow X_r}(x_r) \quad (6)$$

Alg. 1 shows the pseudo-code for the message passing, with the procedure computing message (4) in Alg. 2 and (5) in Alg. 3. Next, we illustrate one run of MPMAP by considering Ex. 4.3 restricted to the formula in Ex. 4.1.

Algorithm 1 MPMAP(Δ, p)

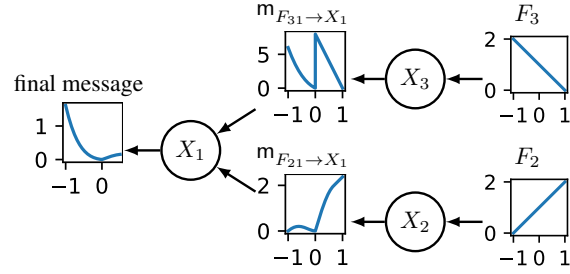
input Δ : SMT formula as in (1), p density as in (2)
output m^* : max. density, $\mathbf{x}^*$: the coordinates
 1: $\mathbf{V}_{\text{up}} \leftarrow$ sort variable nodes according to $\mathcal{G}$
 2: **for each** $X_i \in \mathbf{V}_{\text{up}}$ **do** {upward pass}
 3: gather-msgs($X_i, F_{i, \text{pa}(i)}$) { $m_{X_i \rightarrow F_{i, \text{pa}(i)}}$ }
 4: compute-msgs($F_{i, \text{pa}(i)}, X_{\text{pa}(i)}$) { $m_{F_{i, \text{pa}(i)} \rightarrow X_{\text{pa}(i)}}$ }
 5: **end for**
 6: $m \leftarrow$ gather-root(X_{roots}) {final message}
 7: **return** $\max_x m(x)$, arg max coord $_x m(x)$

Algorithm 2 gather-msgs($X_i, F_{i,j}$)

input X_i : variable, $F_{i,j}$: factor
output $m_{X_i \rightarrow F_{i,j}}$: message
 1: $Q \leftarrow \{m_{F_{j',i} \rightarrow X_i} \mid \forall j' \in \text{neigh}(i) : j' \neq j\} \cup \{m_{F_i \rightarrow X_i}\}$
 2: **return** $\prod_i Q_i$ {point-wise product}

Example 4.6 (MPMAP in action). Consider the factors for Ex. 4.3, and Δ from Ex. 4.1. MPMAP computes the following operations, leading to the computational graph below.

$$\begin{aligned} & \max_{\mathbf{x}} (F_3(x_3) F_{21}(x_2, x_1) F_2(x_2)) (F_{31}(x_3, x_1) F_3(x_3)) \\ &= \max_{x_1} \frac{9}{100} \prod_{i \in \{2,3\}} \underbrace{\max_{x_i} F_{i1}(x_i, x_1) \cdot \underbrace{F_i(x_i)}_{=m_{F_{X_i \rightarrow F_{i1}}}(x_1)}}_{=m_{F_{i1 \rightarrow X_1}}(x_1)} \end{aligned}$$



The main challenge in MPMAP is how to calculate $m_{F_{ij} \rightarrow X_j}(x_j)$, in particular $\max_{x_i} F_{ij}(x_i, x_j) \cdot m_{X_i \rightarrow F_{ij}}(x_i) = \max_{x_i} \llbracket (x_i, x_j) \models \Delta_S \rrbracket \cdot f_{\Omega}(x_i, x_j)$ with $f_{\Omega} \in \Omega$ being the product between $p_S(\mathbf{x}_S)$ and the incoming messages. First, we reduce the problem to a number of symbolic maxima over linear upper and lower bounds which we derive similarly to Zeng et al. (2020a) (lines 2-3 in Alg. 3). The second step is challenging, as symbolic maximization over linear bounds is unexplored. We identify a way to calculate this function explicitly for univariate polynomials (detailed in Appendix A.3), and reduce the bivariate maximization to univariate maximization via $\max_{x_i \in I(x_j)} f_1(x_j) f_2(x_i) = f_1(x_j) \max_{x_i \in I(x_j)} f_2(x_i)$ with $I = [l(x_j), u(x_j)]$, thanks to the properties of our

Algorithm 3 $\text{compute-msgs}(F_{i,j}, X_j)$

input $F_{i,j}$: factor, X_j : variable
output $m_{F_{i,j} \rightarrow X_j}$: message
 1: $m_{F_{i,j} \rightarrow X_j} \leftarrow \text{empty-piecewise}()$
 2: $\mathcal{P} \leftarrow \text{critical-points}(\text{overall-bounds}(m_{X_i \rightarrow F_{i,j}}), \Delta_{i,j})$
 3: $\mathcal{I} \leftarrow \text{intervals-from-points}(\mathcal{P})$
 4: **for** interval $I \in \mathcal{I}$ consistent with formula $\Delta_{i,j}$ **do**
 5: $\mathcal{Q} \leftarrow \text{get-msg-pieces}(m_{X_j \rightarrow F_{i,j}}, I, p_{ij})$
 {enumerates every piece that falls into $X_i \in I$ }
 6: $q'_h \leftarrow \max_{x_i} (q_h, l_h, u_h) \forall (l_h, u_h, q_h) \in \mathcal{Q}$
 7: $m_{F_{i,j} \rightarrow X_j} |_{x_i \in I} \leftarrow \text{max-pieces}(\{q'_h, 0 \leq h < |\mathcal{Q}|\})$
 8: **end for**
 9: **return** $m_{F_{i,j} \rightarrow X_j}$

Algorithm 4 $\text{PAMAP}(\Delta, p, \text{cpenum}, \text{cpopt})$

input Δ : SMT formula, p : density, cpenum : convex polytope enumerator, cpopt : convex polytope optimizer
output m^* : max. density found, $\mathbf{x}^*$: the coordinates
 1: $m^* \leftarrow -\infty$ {current lower bound}
 2: $\mathbf{x}^* \leftarrow \emptyset$ {current best point}
 3: $\text{cpenum.init}(\Delta, p)$
 4: **while** $\text{cpenum.has-next}()$ **do**
 5: $\Pi \leftarrow \text{cpenum.next}()$
 6: $m_\Pi, \mathbf{x}_\Pi \leftarrow \text{cpopt.optimize}(\Pi, p)$
 7: **if** $m_\Pi > m^*$ **then**
 8: $m^*, \mathbf{x}^* \leftarrow m_\Pi, \mathbf{x}_\Pi$
 9: $\text{cpenum.update-lower-bound}(m^*)$
 10: **end if**
 11: **end while**
 12: **return** $m^*, \mathbf{x}^*$

function classes Ω^{PP} and Ω^{PEP} . Finally, we note maximizing a univariate polynomial in Ω^{PP} and Ω^{PEP} can be done with different complexities, depending on the function family as we discuss in Appendix A.5.

5. PAMAP: Scaling Local Convex Optimizers

While our TMC enable tractable $\text{MAP}(\mathcal{LR}, \mathcal{A})$, they might be too restrictive for certain applications. As such, we now introduce a practical algorithm for approximating $\text{MAP}(\mathcal{LR}, \mathcal{A})$ (C2) for an arbitrary SMT($\mathcal{LR}, \mathcal{A}$) formula Δ , and a density p for which an optimization algorithm constrained to a *convex polytope* is available. The algorithm, named PAMAP, is inspired in name and spirit by the WMI-PA algorithm for WMI computation (Morettin et al., 2017; 2019; Spallitta et al., 2022; 2024). It decomposes a potentially non-convex feasible region into convex polytopes, over which constrained optimization can be performed efficiently—and sometimes with guarantees—for many density classes. The procedure, outlined in Alg. 4,

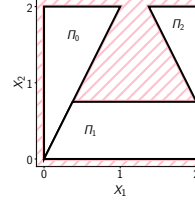


Figure 4. PAMAP can decompose non-convex feasible regions into convex polytopes, as shown for the example in Fig. 1. Note that the partitioning only depends on the constraints Δ . As such, for a conditional density $p(\mathbf{y} | \mathbf{x})$, it needs to be computed only once for all datapoints $\mathbf{x}$ as in the SDD experiments (Sec. 6).

builds on two key components: an enumerator cpenum that partitions the feasible region of Δ into convex polytopes, and a constrained optimizer cpopt to maximize p over each convex polytope. PAMAP maintains the current best solution (lines 1-2), and iteratively considers convex polytopes Π in the partition (lines 4-11). For each Π , it invokes cpopt to find the maximum of p restricted to Π (line 6), updating the best solution accordingly (lines 7-10). Importantly, unlike exact WMI computation, finding the maximum of p does not necessarily require enumerating *all* convex polytopes. Therefore, after updating the best solution, the enumerator is informed of the new lower bound m^* (line 9), allowing it to *prune* polytopes that are known not to contain better solutions. As PAMAP is a family of optimizers, we discuss next how different choices for the base optimizer cpopt and polytope enumeration cpenum can impact performance.

Optimization over convex polytopes. As discussed in Sec. 3, optimization over a single convex polytope is a well-understood problem. Thus, for cpopt in PAMAP, we can leverage off-the-shelf constrained optimizers such as SHGO, which can find optima efficiently, albeit without formal guarantees. Note that numerical optimizers rely on floating-point arithmetic, and as such the returned optimum may be slightly infeasible; nevertheless, such points can be easily projected onto the polytope if needed. For polynomial densities, we can also employ moment-based global optimization (Lasserre, 2001), which provides reliable upper and lower bounds on the global maximum at the cost of a higher runtime, as we quantify empirically in Sec. 6.

Enumeration of convex polytopes. Partitioning the feasible region of an SMT($\mathcal{LR}, \mathcal{A}$) formula into convex polytopes can be reduced to a task known as *AllSMT* (Lahiri et al., 2006; Masina et al., 2025). This involves enumerating truth assignments to linear (in)equalities such that each assignment yields a non-empty convex polytope, and their union covers the feasible region. For this step, we adopt the efficient enumeration techniques used for WMI computation (Spallitta et al., 2024). Often, the density p is defined as a piecewise function, which some constrained optimizers (e.g., Lasserre 2001) cannot handle. In such cases, the structure-aware enumeration by Spallitta et al. (2024) ensures that each enumerated polytope lies entirely within a single piece of p .

Pruning via upper bounds. To prune polytopes that won't contribute to improving the maximum, we can compute an upper bound on p for a given polytope, and if this bound is lower than the current best value m^* , that region can be safely discarded. While computing tight upper bounds is non-trivial in the general case, we can often exploit the specific shape of p to derive efficient bounding techniques. For example, in Appendix C.5.2, we provide additional details on how to compute upper bounds for the case of PAL densities (Kurscheidt et al., 2025) used in our experiments.

Practical limitations and scalability. The primary practical limitation of PAMAP is the potentially exponential growth in the number of convex polytopes w.r.t. the number of linear (in)equalities in the formula. As shown in Sec. 6, however, this worst-case behavior is often not observed in practice, especially when effective pruning strategies are employed. We also note that PAMAP can consume polytopes as they are enumerated, without the need to store the entire set in memory. Hence, memory usage depends only on the implementations of the convex-polytope optimizer and the convex-polytope enumerator. Concerning the optimizer, several numerical optimizers (e.g., Boggs & Tolle 1995; Wales & Doye 1997; Conn et al. 2000; Endres et al. 2018) run in polynomial space. Similarly, a recent polynomial-space AllSMT enumeration algorithm has been proposed (Spallitta et al., 2025).

Approximation guarantees. PAMAP inherits any approximation guarantee from the constrained optimizer used on the enumerated polytopes. For local optimizers, a common requirement is a convex objective, which translates into a concave (log-)density. If the negative (log-)density is now (log-)strongly convex, we can bound the approximation error of the argmax of the density (see section 9.1.2 of Boyd & Vandenberghe (2004)). In contrast, some global optimizers, such as sum-of-squares (SoS) optimization for polynomial densities (Lasserre, 2001), can guarantee exact recovery under suitable conditions but typically exhibit exponential scaling in time or apply only to very restricted function classes.

Relation with OMT-solvers. The idea of decomposing the feasible region into convex polytopes is reminiscent of the lazy OMT loop (Bjørner et al., 2015; Sebastiani & Tomasi, 2015). There are, however, several key differences that make PAMAP more practical for solving MAP($\mathcal{LR}\mathcal{A}$) problems. First, current OMT-solvers only support polynomial densities, whereas the modularity of PAMAP allows handling any density which admits a constrained optimizer. E.g. mixtures of Gaussians, and also stochastic densities estimated via Monte Carlo sampling, as we show in Sec. 6. Second, even restricting to polynomial densities, OMT-solvers are not specialized for the case of SMT($\mathcal{LR}\mathcal{A}$) constraints, and thus address a computationally harder problem. In contrast,

PAMAP decouples the enumeration of convex polytopes from the optimization step, combining efficient enumeration for SMT($\mathcal{LR}\mathcal{A}$) and specialized optimizers and pruning techniques for convex polytopes. Finally, this decoupling allows for straightforward parallelization, as different polytopes can be optimized concurrently.

6. Experiments

We now empirically evaluate our MPMAP and PAMAP on several real-world and synthetic benchmarks (C3). Specifically, we aim to answer these research questions: **Q1**) How well does MPMAP scale and how does it compare to approximate optimizers for exact constrained MAP? **Q2**) How does PAMAP trade off solution quality and runtime on real-world problems? We describe our setup, baselines, and results for both algorithms next. Experimental settings are detailed in Appendix C and the code to reproduce experiments can be found at github.com/april-tools/constrained-map.

Baselines and comparison. We compare against OMT-solvers for non-linear real arithmetic, namely OptiMathSAT (Sebastiani & Trentin, 2020) and CDCL-OCAC (Jia et al., 2025). As a first baseline, we compare against the classical ADAM optimizer (Kingma & Ba, 2015), maximizing $p(\mathbf{x})$ without considering the constraints Δ . This approach has two limitations: first, it may return infeasible solutions that do not satisfy Δ ; second, it is prone to getting stuck in local optima, especially in high-dimensional, non-convex landscapes. Hence, as a side contribution (C2), we introduce a more competitive baseline PCADAM, a *particle-based, constraints-aware version of ADAM*. PCADAM maintains a set of N particles (i.e., candidate solutions) that are iteratively updated using ADAM. Crucially, at each iteration, we update the best result found so far among all particles that *satisfy* the constraints Δ . Appendix B provides further details. Finally, we also compare against DL2 (Fischer et al., 2019), where we optimize the soft-constrained objective using ADAM with multiple random restarts.

Q1) Scalability of MPMAP. To benchmark our message-passing scheme, we follow Zeng et al. (2020a) and generate three different kinds of tree-shaped problems in varying dimensions and diameters: *STAR* (star-shaped), *SNOW* (ternary-tree), or *PATH* (linear-chain) trees, which represent real-world applications like phylogenetic trees (Nei & Kumar, 2000) and fault tree analysis (Vesely et al., 1981). For each shape, we sample random N -variable SMT formulas and couple them with random, unnormalized densities in Ω^{PP} (Sec. 4). Appendix C.2 further details the setup. In total, this procedure yields 1059 benchmark instances. Examples of these instances (for 2 dimensions) are visualized in Appendix C.2 in Figs. 9, 10 and 11, highlighting both the non-convex constraints and the multimodal densities which render the optimization problem challenging.

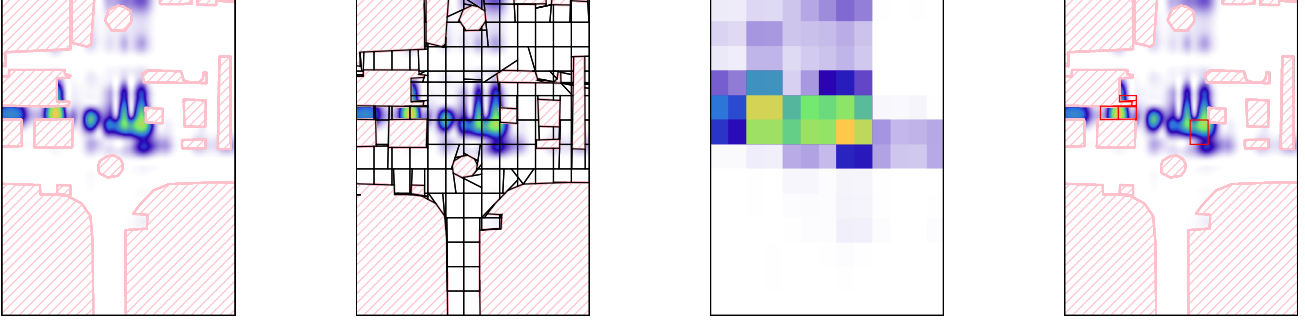


Figure 5. On SDD, our upper-bound pruning strategy drastically reduces the number of polytopes optimized by PAMAP. Left: constrained density for an agent’s next position. Center-left: 257 convex polytopes considered by PAMAP without upper-bound-based pruning. Center-right: piecewise-constant upper bound. Right: only 9 convex polytopes considered by PAMAP with pruning. More examples in Fig. 13 in Appendix C.5.2.

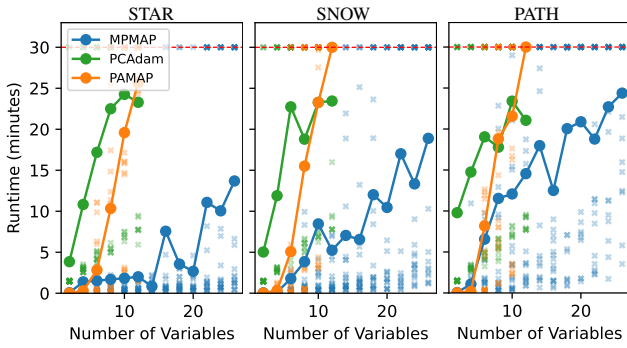


Figure 6. When $\text{MAP}(\mathcal{LR})$ has tree-structure, MPMAP outperforms competitors such as PAMAP and PCADAM on graphs with different diameter, including PATH graphs with maximal diameter $d=N-1$, where the worst-case complexity of MPMAP would scale exponentially. Details in Appendix C.2.

We compare three algorithms on these problems: MPMAP, PAMAP using SHGO (Endres et al., 2018) as optimizer, and PCADAM. We do not compare to the OMT-solvers in this experiment due to their limited scalability. Since both PAMAP and PCADAM are optimizers without optimality guarantees, they can return arbitrarily fast but potentially very poor solutions. To control for solution quality, we therefore introduce a simple grid-search on the enumerated polytopes that these methods must outperform (details in Appendix C.3) until either the time-budget is exhausted or they surpass the baseline.

Fig. 6 shows the results, demonstrating the superior scalability of exact message passing compared to approximate optimizers. Here, we scale PAMAP and PCADAM only up to dimension 12, as runtimes increase rapidly beyond this point and render the experiments prohibitively long. Notably, the advantage of MPMAP persists for PATH problems, which have maximal graph diameter $d=N-1$ and for which the theoretical complexity of MPMAP scales exponentially in d (Theorem A.8). Detailed results are in Tables 2, 3 and 4.

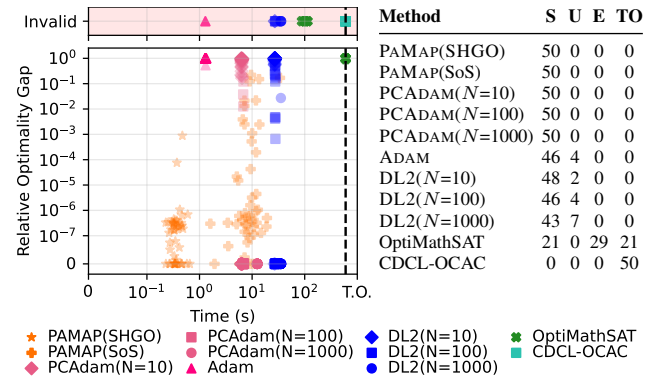


Figure 7. On SDD, PAMAP outperforms other solvers in both time and solution quality. Left: Scatter plot of runtime vs. relative optimality gap. Right: Summary table reporting (S) instances solved with a point satisfying Δ , (U) instances where the returned point does not satisfy Δ , (E) instances terminated with an error, and (TO) number of timeouts (600s). Note that OptiMathSAT is anytime and returns the best solution found before timeout.

Q2) Trajectory prediction with PAMAP. We evaluate PAMAP on a first real-world application, and we consider the Stanford drone dataset (SDD) (Robicquet et al., 2016) a collection of drone-captured trajectories of multiple agents. The agents can only move in walkable areas, modeled as SMT constraints over the 2D space. Following Kurscheidt et al. (2025), we learned a predictive density for an agent’s future position, conditioned on its past trajectory and the scene layout. We then generated 50 test instances by sampling different agent trajectories. In Fig. 5 we show an example of such a density. The figure also shows an execution of PAMAP on this instance, demonstrating how the computation of upper bounds is crucial for pruning the vast majority of the polytopes, and thus improving efficiency. In Appendix C.5.2 we show further examples (Fig. 13) and provide details on upper bound computation.

We run PAMAP using two different optimizers: a numerical optimizer (SHGO), and an optimizer based on the SoS-

Moment hierarchy (see Sec. 3). We compare it against ADAM, PCADAM and DL2 with different number of particles N , OptiMathSAT run in anytime mode, and CDCL-OCAC (see Appendix C.5.2). Results are shown in Fig. 7. Here solution quality is measured in terms of relative optimality gap, computed as $\max(0, v^* - v)/v^*$, between the value found v and the best known value v^* computed via a grid-search. From the plot, we see that PAMAP(SHGO) achieves the best trade-off between runtime and solution quality. PCADAM can be competitive in terms of solution quality, but only if enough particles are used, leading to significantly higher runtimes. PAMAP(SoS), here used with a relaxation order of 7, is significantly slower than the SHGO variant. ADAM and DL2 are not reliable at finding good, feasible solutions. OMT-solvers struggle in this setting: CDCL-OCAC solved no instances, while OptiMathSAT found only poor-quality solutions or reported errors.

Q2) Data imputation of a VAE with PAMAP. As a second real-world application, we apply PAMAP to data imputation with a density encoded in a VAE (Kingma & Welling, 2014). In this experiment, we use the *House-Sales* tabular dataset with the provided constraints from Stoian & Giunchiglia (2025). We train an unconstrained VAE on the train-dataset and mask random features on the test-dataset. Constraints here are in the form of SMT rules over features of a house, such as squared meters, number of rooms and cost. To evaluate the benefit of constrained imputation, masking is restricted to variables for which constraints are available. For each of 400 test samples, every eligible feature is masked independently with probability 10%, and we compute a MAP estimate over the masked variables subject to the constraints. As we cannot compute a VAE density exactly, we approximate it via Monte Carlo samples from the latent prior $\mathbf{z}$, so $p(\mathbf{x}_m) \approx 1/N \sum_{\mathbf{z} \sim \mathcal{N}(0, I)} p_{\text{dec}}(\mathbf{x}_m, \mathbf{x}_o | \mathbf{z})$, where $\mathbf{x}_m$ (resp. $\mathbf{x}_o$) are the missing (resp. observed) features, p_{dec} is the decoder architecture, defined as a neural network outputting an isotropic Gaussian, and $\mathbf{z}$ is the latent code of the VAE. We measure the relative error via $\text{RE}(\mathbf{x}^*) = \text{avg}(|\mathbf{x}^* - \mathbf{x}_{gt}| / (|\mathbf{x}_{gt}| + 1))$, where $\mathbf{x}_{gt}$ is the ground truth value for the missing features that are predicted as $\mathbf{x}^*$ by PAMAP. To demonstrate the advantage of combining the VAE prior with domain knowledge constraints in data imputation, we compare PAMAP against a particle version of ADAM with $N \in \{10, 100\}$. For PAMAP, we need a convex-polytope optimizer that can handle a stochastic objective. Since the numerical optimizers we used in the trajectory experiment are not designed for this, we use our PCADAM as convex-polytope optimizer within PAMAP. Despite the stochastic objective, PAMAP substantially improves imputation accuracy: averaged over the dataset, we obtain a lower error and outperform particle ADAM in 78% of samples in our experiment (Fig. 8), showing that MAP inference under non-convex constraints is both practical and

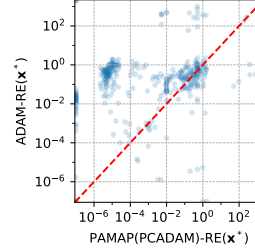


Figure 8. Combining the VAE prior with domain knowledge constraints, PAMAP substantially surpasses the unconstrained prediction of ADAM. We compare the relative error on the data-imputation task. Further statistics are reported in Appendix C.6.3.

effective for real-world tabular data imputation.

Table 1. PAMAP outperforms both domain-knowledge-aware and unconstrained baselines, even when competing methods use more particles. We compare the relative error $\text{RE}(\mathbf{x}^*)$ for PAMAP(PCADAM) to a particle-enhanced version of ADAM and the soft-constraint penalty method DL2. Further results can be found in the appendix in Table 5.

Method	PAMAP	ADAM	ADAM	DL2	DL2
Particles	10	10	100	10	100
$\text{RE}(\mathbf{x}^*)$	0.22	33.12	25.85	12.22	16.78

In addition to comparing against particle ADAM, we investigate the benefit of enumerating the feasible regions over soft-constrained approaches and compare against DL2 (Fischer et al., 2019). Unfortunately, OMT solvers do not support these densities, preventing a direct comparison in this setting. We show the results in Table 1, where we outperform the relative error of the baselines even when the competing methods use more particles. Further details, including timings, are provided in Appendix C.6.

7. Conclusion

In this work, we traced the theoretical and practical foundations of MAP inference in the presence of non-convex SMT($\mathcal{LR}$) constraints and non-log-concave densities. To this end, we introduced two new solvers that substantially advance the SoTA for this challenging inference task. The first, MPMAP can deal with a new and non-trivial fragment of tractable MAP($\mathcal{LR}$) problems and can advance our understanding of the complexity of this task, as well as help us design further approximation schemes based on it (Zeng et al., 2020b). The second, PAMAP, is a practical and modular framework combining SMT-based convex polytope enumeration, with constrained optimization over each such polytope. In our rigorous experiments, we showed not only that our solvers can achieve a better trade off between solution quality and execution time, but also that they can be used to perform constrained inference in conjunction with black-box deep generative models. In the future, we plan to use them to design reliable generative models that satisfy non-convex constraints by design (van Krieken et al., 2025; Marconato et al., 2025).

Impact Statement

We advance both theoretical and empirical understanding of MAP Inference under Non-Convex Constraints. As a general contribution to the field of machine learning, there are many potential societal consequences of our work. Here, we want to highlight one: As constraints enable a more detailed, explicit control over the black-box prediction of machine learning models, they can be used to encode rules and knowledge of the machine learning practitioner. This can be both abused by consciously or unconsciously encoding biases, but can also be used to encode beneficial constraints such as fairness constraints. This example illustrates that constraints in MAP inference provide a mechanism to guide model behavior, with both potential risks and benefits.

Reproducibility Statement

The source code and the instructions to reproduce our results can be found at: github.com/april-tools/constrained-map. A detailed description of all experimental settings is given in Appendix C.

Contribution

GM, LK, AV and RS conceived the initial idea of the paper. LK is responsible for all theoretical contributions, illustrations, algorithms and the implementation related to MPMAP. GM is responsible for all theoretical contributions, illustrations, algorithms and the implementation related to PAMAP. LK conceived and implemented the experiment testing the scalability of MPMAP and the missing-value experiment for PAMAP, while GM conceived and implemented the experiment of trajectory prediction with PAMAP. GM and LK wrote the paper with help from AV and feedback from RS. AV supervised all phases of the project.

Acknowledgements

We thank Dylan Ponsford for valuable feedback on the draft. AV was supported by the “UNREAL: Unified Reasoning Layer for Trustworthy ML” project (EP/Y023838/1) selected by the ERC and funded by UKRI EPSRC. RS was partially supported by the project “AI@TN” funded by the Autonomous Province of Trento. RS was partially supported by the MUR PNRR project FAIR - Future AI Research (PE00000013) funded by the NextGenerationEU; by the NRRP, Mission 4 Component 2 Investment 1.4, by the European Union — NextGenerationEU (proj. nr. CN 00000013); and by the TANGO project funded by the EU Horizon Europe research and innovation program under GA No 101120763, funded by the European Union. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European

Union, the European Health and Digital Executive Agency (HaDEA) or The European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

References

- Akritas, A. G. and Strzebonski, A. W. A comparative study of two real root isolation methods. *Nonlinear Analysis: Modelling and Control*, 10(4):297–304, Oct. 2005. doi: 10.15388/NA.2005.10.4.15110. URL <https://www.journals.vu.lt/nonlinear-analysis/article/view/15110>.
- Apostol, T. M. *Calculus, Volume 1*. John Wiley & Sons, 1991. Taylor’s theorem with remainder (Peano form).
- Arnon, D. S., Collins, G. E., and McCallum, S. Cylindrical Algebraic Decomposition I: The Basic Algorithm. *SIAM Journal on Computing*, 13(4):865–877, 1984. ISSN 0097-5397. doi: 10.1137/0213054.
- Barrett, C., Sebastiani, R., Seshia, S. A., and Tinelli, C. Satisfiability Modulo Theories. In *Handbook of Satisfiability*, volume 336 of *FAIA*, pp. 1267–1329. IOS Press, 2 edition, 2021. ISBN 978-1-64368-160-3 978-1-64368-161-0. doi: 10.3233/FAIA201017.
- Belle, V., Passerini, A., and Van den Broeck, G. Probabilistic inference in hybrid domains by weighted model integration. In *Proceedings of the Twenty-Fourth International Joint Conference on Artificial Intelligence, IJCAI 2015, Buenos Aires, Argentina, July 25-31, 2015*, pp. 2770–2776. IJCAI Inc, 2015.
- Bigarella, F., Cimatti, A., Griggio, A., Irfan, A., Jonáš, M., Roveri, M., Sebastiani, R., and Trentin, P. Optimization Modulo Non-linear Arithmetic via Incremental Linearization. In *Frontiers of Combining Systems*, LNCS, pp. 213–231. Springer, 2021. ISBN 978-3-030-86205-3. doi: 10.1007/978-3-030-86205-3_12.
- Bishop, C. M. *Pattern Recognition and Machine Learning*. Springer-Verlag, Berlin, Heidelberg, 2006. ISBN 0387310738.
- Bjørner, N., Phan, A.-D., and Fleckenstein, L. νZ - An Optimizing SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems*, LNCS, pp. 194–199. Springer, 2015. ISBN 978-3-662-46681-0. doi: 10.1007/978-3-662-46681-0_14.
- Boggs, P. T. and Tolle, J. W. Sequential Quadratic Programming. *Acta Numerica*, 4:1–51, 1995. ISSN 1474-0508, 0962-4929. doi: 10.1017/S0962492900002518.

- Bortolotti, S., Marconato, E., Carraro, T., Morettin, P., van Krieken, E., Vergari, A., Teso, S., and Passerini, A. A neuro-symbolic benchmark suite for concept quality and reasoning shortcuts. *Advances in neural information processing systems*, 37:115861–115905, 2024.
- Boyd, S. and Vandenberghe, L. *Convex optimization*. Cambridge university press, 2004.
- Cheng, C., Han, B., Maddix, D. C., Ansari, A. F., Stuart, A., Mahoney, M. W., and Wang, Y. Gradient-free generation for hard-constrained systems. *arXiv preprint arXiv:2412.01786*, 2024.
- Chin, E. B., Lasserre, J. B., and Sukumar, N. Numerical integration of homogeneous functions on convex and nonconvex polygons and polyhedra. *Computational Mechanics*, 56(6):967–981, 2015. ISSN 1432-0924. doi: 10.1007/s00466-015-1213-7.
- Choi, Y., Vergari, A., and Van den Broeck, G. Probabilistic circuits: A unifying framework for tractable probabilistic modeling. Technical report, University of California, Los Angeles (UCLA), 2020.
- Collins, G. E. Quantifier elimination for real closed fields by cylindrical algebraic decomposition. In *Automata Theory and Formal Languages*, pp. 134–183. Springer, 1975. ISBN 978-3-540-37923-2. doi: 10.1007/3-540-07407-4_17.
- Conn, A. R., Gould, N. I. M., and Toint, P. L. Part III Trust-Region Methods for Constrained Optimization with Convex Constraints. In *Trust Region Methods*, MOS-SIAM Series on Optimization, pp. 439–439. Society for Industrial and Applied Mathematics, 2000. ISBN 978-0-89871-460-9. doi: 10.1137/1.9780898719857.pt3.
- Cunningham, J. P., Hennig, P., and Lacoste-Julien, S. Approximate gaussian integration using expectation propagation. *arXiv: Machine Learning*, 2011. URL <https://api.semanticscholar.org/CorpusID:88518414>.
- Dantzig, G. B. Linear programming. *Operations research*, 50(1):42–47, 2002.
- De Smet, L., Dos Martires, P. Z., Manhaeve, R., Marra, G., Kimmig, A., and De Readt, L. Neural probabilistic logic programming in discrete-continuous domains. In *Uncertainty in Artificial Intelligence*, pp. 529–538. PMLR, 2023.
- Diligenti, M., Gori, M., and Saccà, C. Semantic-based regularization for learning and inference. *Artif. Intell.*, 244:143–165, 2017. doi: 10.1016/J.ARTINT.2015.08.011. URL <https://doi.org/10.1016/j.artint.2015.08.011>.
- Doss, C. R. and Wellner, J. A. Inference for the mode of a log-concave density. *The Annals of Statistics*, 47(5): 2950–2976, 2019.
- Endres, S. C., Sandrock, C., and Focke, W. W. A simplicial homology algorithm for Lipschitz optimisation. *J Glob Optim*, 72(2):181–217, 2018. ISSN 1573-2916. doi: 10.1007/s10898-018-0645-y.
- Fischer, M., Balunovic, M., Drachsler-Cohen, D., Gehr, T., Zhang, C., and Vechev, M. T. DL2: training and querying neural networks with logic. In Chaudhuri, K. and Salakhutdinov, R. (eds.), *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, Proceedings of Machine Learning Research, pp. 1931–1941. PMLR, 2019. URL <http://proceedings.mlr.press/v97/fischer19a.html>.
- Ghandi, S., Quost, B., and de Campos, C. Probabilistic circuits with constraints via convex optimization. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pp. 161–177. Springer, 2024.
- Giunchiglia, E., Stoian, M. C., Khan, S., Cuzzolin, F., and Lukasiewicz, T. Road-r: the autonomous driving dataset with logical requirements. *Machine Learning*, 112(9): 3261–3291, 2023.
- González, M., Almansa, A., and Tan, P. Solving inverse problems by joint posterior maximization with autoencoding prior. *SIAM Journal on Imaging Sciences*, 15(2): 822–859, 2022.
- Grivas, A., Vergari, A., and Lopez, A. Taming the sigmoid bottleneck: Provably argmaxable sparse multi-label classification. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 12208–12216, 2024.
- Hansen, D., Maddix, D. C., Alizadeh, S., Gupta, G., and Mahoney, M. W. Learning physical models that can respect conservation laws. In *International Conference on Machine Learning*, pp. 12469–12510. PMLR, 2023.
- Jaggi, M. Revisiting frank-wolfe: Projection-free sparse convex optimization. In *International conference on machine learning*, pp. 427–435. PMLR, 2013.
- Jeong, J., Sanner, S., and Kumar, A. A mixed-integer linear programming reduction of disjoint bilinear programs via symbolic variable elimination. In Cire, A. A. (ed.), *Integration of Constraint Programming, Artificial Intelligence, and Operations Research*, pp. 79–95, Cham, 2023. Springer Nature Switzerland. ISBN 978-3-031-33271-5.

- Jia, F., Dong, Y., Han, R., Huang, P., Liu, M., Ma, F., and Zhang, J. A Complete Algorithm for Optimization Modulo Nonlinear Real Arithmetic. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(11):11255–11263, 2025. ISSN 2374-3468. doi: 10.1609/aaai.v39i11.33224.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. In Bengio, Y. and LeCun, Y. (eds.), *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015. URL <http://arxiv.org/abs/1412.6980>.
- Kingma, D. P. and Welling, M. Auto-encoding variational bayes. In *ICLR*, 2014.
- Kurscheidt, L., Morettin, P., Sebastiani, R., Passerini, A., and Vergari, A. A Probabilistic Neuro-symbolic Layer for Algebraic Constraint Satisfaction. In *The 41st Conference on Uncertainty in Artificial Intelligence*, 2025.
- Lahiri, S. K., Nieuwenhuis, R., and Oliveras, A. SMT Techniques for Fast Predicate Abstraction. In *Computer Aided Verification*, LNCS, pp. 424–437. Springer, 2006. ISBN 978-3-540-37406-0. doi: 10.1007/11817963_39.
- Lasserre, J. B. Global Optimization with Polynomials and the Problem of Moments. *SIAM Journal on Optimization*, 11(3):796–817, 2001. ISSN 1052-6234. doi: 10.1137/S1052623400366802.
- Li, Y., Albarghouthi, A., Kincaid, Z., Gurfinkel, A., and Chechik, M. Symbolic optimization with SMT solvers. In *41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pp. 607–618. ACM, 2014. ISBN 978-1-4503-2544-8. doi: 10.1145/2535838.2535857.
- Loconte, L., Sladek, A. M., Mengel, S., Trapp, M., Solin, A., Gillis, N., and Vergari, A. Subtractive mixture models via squaring: Representation and learning. In *The Twelfth International Conference on Learning Representations*, 2024.
- Loconte, L., Javaloy, A., and Vergari, A. How to square tensor networks and circuits without squaring them. *ArXiv preprint*, abs/2512.17090, 2025a. URL <https://arxiv.org/abs/2512.17090>.
- Loconte, L., Mengel, S., and Vergari, A. Sum of squares circuits. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 19077–19085, 2025b.
- Marconato, E., Bortolotti, S., van Krieken, E., Morettin, P., Umili, E., Vergari, A., Tsamoura, E., Passerini, A., and Teso, S. Symbol grounding in neuro-symbolic ai: A gentle introduction to reasoning shortcuts. *arXiv preprint arXiv:2510.14538*, 2025.
- Marinescu, R. and Dechter, R. And/or tree search for constraint optimization. In *6th international workshop on preferences and soft constraints*, 2004.
- Martins, A. F., Figueiredo, M. A., Aguiar, P. M., Smith, N. A., and Xing, E. P. An augmented lagrangian approach to constrained map inference. In *ICML*, volume 2, pp. 2, 2011.
- Masina, G., Spallitta, G., and Sebastiani, R. On CNF Conversion for SAT and SMT Enumeration. *Journal of Artificial Intelligence Research*, 83, 2025. ISSN 1076-9757. doi: 10.1613/jair.1.16870.
- Morettin, P., Passerini, A., and Sebastiani, R. Efficient Weighted Model Integration via SMT-Based Predicate Abstraction. In *26th International Joint Conference on Artificial Intelligence*, pp. 720–728. International Joint Conferences on Artificial Intelligence Organization, 2017. ISBN 978-0-9992411-0-3. doi: 10.24963/ijcai.2017/100.
- Morettin, P., Passerini, A., and Sebastiani, R. Advanced SMT techniques for Weighted Model Integration. *Artificial Intelligence*, 275(C):1–27, 2019. ISSN 00043702. doi: 10.1016/j.artint.2019.04.003.
- Narasimhan, S. S., Agarwal, S., Akcin, O., Sanghavi, S., and Chinchali, S. P. Time weaver: A conditional time series generation model. In *International Conference on Machine Learning*, pp. 37293–37320. PMLR, 2024.
- Nei, M. and Kumar, S. *Molecular Evolution and Phylogenetics*. Oxford University Press, 2000. ISBN 9780195350517. URL <https://books.google.co.uk/books?id=hcPSag2pn9IC>.
- Nieuwenhuis, R. and Oliveras, A. On SAT Modulo Theories and Optimization Problems. In *9th International Conference on Theory and Applications of Satisfiability Testing*, LNCS, pp. 156–169. Springer, 2006. ISBN 978-3-540-37207-3. doi: 10.1007/11814948_18.
- Nocedal, J. and Wright, S. J. *Numerical Optimization*. Springer Series in Operations Research and Financial Engineering. Springer, 2 edition, 2006. ISBN 978-0-387-30303-1. doi: 10.1007/978-0-387-40065-5.
- Powers, V. and Wörmann, T. An algorithm for sums of squares of real polynomials. *Journal of Pure and Applied Algebra*, 127:99–104, 1998.
- Robicquet, A., Sadeghian, A., Alahi, A., and Savarese, S. Learning Social Etiquette: Human Trajectory Understanding In Crowded Scenes. In *Computer Vision – ECCV 2016*, pp. 549–565. Springer, 2016. ISBN 978-3-319-46484-8. doi: 10.1007/978-3-319-46484-8_33.

- Sahni, S. Computationally Related Problems. *SIAM Journal on Computing*, 3(4):262–279, 1974. ISSN 0097-5397, 1095-7111. doi: 10.1137/0203021.
- Sanner, S. and McAllester, D. Affine algebraic decision diagrams (aadds) and their application to structured probabilistic inference. In *Proceedings of the 19th International Joint Conference on Artificial Intelligence, IJCAI’05*, pp. 1384–1390, San Francisco, CA, USA, 2005. Morgan Kaufmann Publishers Inc.
- Schönhage, A. The fundamental theorem of algebra in terms of computational complexity. *Manuscript. Univ. of Tübingen, Germany*, 1982.
- Sebastiani, R. and Tomasi, S. Optimization in SMT with LA(Q) Cost Functions. In *International Joint Conference on Automated Reasoning*, volume 7364 of *LNCS*, pp. 484–498. Springer, 2012. ISBN 978-3-642-31364-6. doi: 10.1007/978-3-642-31365-3_38.
- Sebastiani, R. and Tomasi, S. Optimization Modulo Theories with Linear Rational Costs. *ACM Transactions on Computational Logic*, 16(2):12:1–12:43, 2015. ISSN 1529-3785. doi: 10.1145/2699915.
- Sebastiani, R. and Trentin, P. OptiMathSAT: A Tool for Optimization Modulo Theories. *Journal of Automated Reasoning*, 64(3):423–460, 2020. ISSN 1573-0670. doi: 10.1007/s10817-018-09508-6.
- Smet, L. D., Martires, P. Z. D., Manhaeve, R., Marra, G., Kimmig, A., and Raedt, L. D. Neural probabilistic logic programming in discrete-continuous domains. In Evans, R. J. and Shpitser, I. (eds.), *Uncertainty in Artificial Intelligence, UAI 2023, July 31 - 4 August 2023, Pittsburgh, PA, USA*, Proceedings of Machine Learning Research, pp. 529–538. PMLR, 2023. URL <https://proceedings.mlr.press/v216/de-smet23a.html>.
- Spallitta, G., Masina, G., Morettin, P., Passerini, A., and Sebastiani, R. SMT-based Weighted Model Integration with Structure Awareness. In *38th Conference on Uncertainty in Artificial Intelligence*, volume 180, pp. 1876–1885. PMLR, 2022.
- Spallitta, G., Masina, G., Morettin, P., Passerini, A., and Sebastiani, R. Enhancing SMT-based Weighted Model Integration by Structure Awareness. *Artificial Intelligence*, 328:104067, 2024. ISSN 0004-3702. doi: 10.1016/j.artint.2024.104067.
- Spallitta, G., Sebastiani, R., and Biere, A. Disjoint projected enumeration for SAT and SMT without blocking clauses. *Artificial Intelligence*, 345:104346, 2025. ISSN 0004-3702. doi: 10.1016/j.artint.2025.104346.
- Stewart, R. and Ermon, S. Label-free supervision of neural networks with physics and domain knowledge. *CoRR*, abs/1609.05566, 2016. URL <http://arxiv.org/abs/1609.05566>.
- Stoian, M. C. and Giunchiglia, E. Beyond the convexity assumption: Realistic tabular data generation under quantifier-free real linear constraints. In *The Thirteenth International Conference on Learning Representations*, 2025.
- van Krieken, E., Minervini, P., Ponti, E., and Vergari, A. Neurosymbolic diffusion models. In *NeurIPS*, 2025.
- Vergari, A., Choi, Y., Liu, A., Teso, S., and Van den Broeck, G. A compositional atlas of tractable circuit operations for probabilistic inference. *Advances in Neural Information Processing Systems*, 34, 2021.
- Vesely, W., of Systems, U. N. R. C. D., and Research, R. *Fault Tree Handbook*. Number v. 88 in *Fault Tree Handbook*. Systems and Reliability Research, Office of Nuclear Regulatory Research, U.S. Nuclear Regulatory Commission, 1981. ISBN 9780160055829. URL <https://books.google.co.uk/books?id=x9t9qjLFm9sC>.
- Vincent, A. J. H. *Note sur la résolution des équations numériques*. 1834.
- Wales, D. J. and Doye, J. P. K. Global Optimization by Basin-Hopping and the Lowest Energy Structures of Lennard-Jones Clusters Containing up to 110 Atoms. *The Journal of Physical Chemistry A*, 101(28):5111–5116, 1997. ISSN 1089-5639, 1520-5215. doi: 10.1021/jp970984n.
- Wolfram Research. Exact Global Optimization. <https://reference.wolfram.com/language/tutorial/ConstrainedOptimizationExact.html.en>. Accessed: 2025-11-12.
- Xu, J., Zhang, Z., Friedman, T., Liang, Y., and den Broeck, G. V. A semantic loss function for deep learning with symbolic knowledge. *CoRR*, abs/1711.11157, 2017. URL <http://arxiv.org/abs/1711.11157>.
- Xu, L., Skoularidou, M., Cuesta-Infante, A., and Veeramachaneni, K. Modeling tabular data using conditional GAN. In Wallach, H. M., Larochelle, H., Beygelzimer, A., d’Alché-Buc, F., Fox, E. B., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pp. 7333–7343, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/>

254ed7d2de3b23ab10936522dd547b78-Abstract.html.

Ye, Z., Say, B., and Sanner, S. Symbolic bucket elimination for piecewise continuous constrained optimization. In van Hoeve, W.-J. (ed.), *Integration of Constraint Programming, Artificial Intelligence, and Operations Research*, pp. 585–594, Cham, 2018. Springer International Publishing. ISBN 978-3-319-93031-2.

Zamani, Z., Sanner, S., and Fang, C. Symbolic dynamic programming for continuous state and action mdps. *Proceedings of the AAAI Conference on Artificial Intelligence*, 26(1):1839–1845, Sep. 2021. doi: 10.1609/aaai.v26i1.8372. URL <https://ojs.aaai.org/index.php/AAAI/article/view/8372>.

Zeng, Z. and den Broeck, G. V. Efficient search-based weighted model integration. In Globerson, A. and Silva, R. (eds.), *Proceedings of the Thirty-Fifth Conference on Uncertainty in Artificial Intelligence, UAI 2019, Tel Aviv, Israel, July 22-25, 2019*, volume 115 of *Proceedings of Machine Learning Research*, pp. 175–185. AUAI Press, 2019. URL <http://proceedings.mlr.press/v115/zeng20a.html>.

Zeng, Z., Morettin, P., Yan, F., Vergari, A., and Van den Broeck, G. Scaling up hybrid probabilistic inference with logical and arithmetic constraints via message passing. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, jul 2020a. URL <http://starai.cs.ucla.edu/papers/ZengICML20.pdf>.

Zeng, Z., Morettin, P., Yan, F., Vergari, A., and Van den Broeck, G. Probabilistic inference with algebraic constraints: Theoretical limits and practical approximations. *Advances in Neural Information Processing Systems*, 33: 11564–11575, 2020b.

Zeng, Z., Morettin, P., Yan, F., Vergari, A., and Van den Broeck, G. Relax, compensate and then integrate. In *Proceedings of the ECML-PKDD Workshop on Deep Continuous-Discrete Machine Learning (DeCoDeML)*, 2020c. URL <http://starai.cs.ucla.edu/papers/ZengDECODEML20.pdf>.

A. A Scalable Message-Passing Algorithm For Constrained MAP

A.1. Incomparability of tractable MAP and integration over SMT formulas

While the $\text{MAP}(\mathcal{LR})$ problem is reminiscent of computing the weighted model integral, there is a major difference to the WMI. Our fundamental operation is not integrating out a variable like $\int_{l(x_j)}^{u(x_j)} q(x_j, x_i) dx_i$, but maximizing out a variable like $\max_{x_i \in [l(x_j), u(x_j)]} q(x_j, x_i)$. It is important to note that l and u are symbolic bounds, so affine functions in x_j . In order to have a general algorithm for the maximization in Appendix A.3, we have to detail the function class we are actually able to handle and how they differ from the class investigated by Zeng et al. (2020a). As we previously mentioned, we are considering two main families of functions, piecewise polynomials Ω^{PP} and piecewise exponentiation polynomials Ω^{PEP} .

As we can reduce the task of maximizing exponentiated polynomials to the task of maximizing polynomials in general by going into log-space before maximizing, we will first consider the task of maximizing piecewise polynomials Ω^{PP} . Therefore, when thinking about how to maximize out a variable of a piecewise polynomial, the first question to ask is which kind of $q(x_j, x_i)$ stays a piecewise polynomial after maximizing out x_i . This is required in order to compute our message-passing algorithm by recursing into the tree. Unfortunately, while Zeng et al. (2020a) is able to handle general polynomials over two variables, our choice of function classes is more limited. The issue is that even a simple, general cubic polynomial in two variables already falls outside our polynomial function space when taking the symbolic max; for example, using constant upper and lower bounds, we have:

$$\begin{aligned} h'(x) &= \max_{y \in [0,1]} h(x, y) \\ &= \max_{y \in [0,1]} y^3 - 1.5xy^2 + xy \\ &= h(x, y) \Big|_{y=\frac{x}{2} - \frac{\sqrt{9x^2-12x}}{6}} \notin \mathbb{R}[x] \end{aligned}$$

In this derivation, we assume $x \geq 2$, so $\text{dom}(h') = [2, \infty)$. We therefore focus on the function class of piecewise separable non-negative polynomials Ω^{PP} , so piecewise polynomials where each piece factors as a product of univariate polynomials:

$$p_S(\mathbf{x}_S) = \begin{cases} \prod_{s \in S} q_1^s(x_s) & \text{if } \mathbf{x}_S \models \Phi_1, \\ \vdots & \\ \prod_{s \in S} q_k^s(x_s) & \text{if } \mathbf{x}_S \models \Phi_k, \end{cases} \iff p_S(\mathbf{x}_S) \in \Omega^{\text{PP}} \quad (7)$$

Here, q_i^s is a univariate polynomial over the variable x_s , and Φ_i a conjunction of literals over $\mathbf{x}_S$, so a polytope. We require Φ_k to be non-overlapping. This construction allows us to push the max-operator past the term dependent on other variable while the piecewise-nature retains flexibility. As we later enumerate the pieces in Alg. 5, we can switch it with the maximum per piece:

$$\begin{aligned} h(x) &= \max_{y \in [l(x), u(x)]} f(x, y) \cdot q(y) \\ &= \max_{y \in [l(x), u(x)]} f_1(x) \cdot f_2(y)q(y) \\ &= f_1(x) \cdot \underbrace{\max_{y \in [l(x), u(x)]} f_2(y)q(y)}_{\text{max-out algorithm (Appendix A.3)}} \end{aligned}$$

In Appendix A.3 we derive an algorithm to maximize a univariate, piecewise polynomial under symbolic linear upper and lower bounds, which is the only missing piece we need in order to explicitly compute h .

The construction of Ω^{PEP} is analogous except that q_i^s are univariate exponentiated polynomials. It is important to note that Ω^{PEP} is more general than Zeng et al. (2020a) derived for the integral, where the integration can only be performed over exponentiated linear functions, but we allow the addition of univariate polynomials of arbitrary degree in the exponent (or as a product before exponentiating).

A.2. The Message-Passing Algorithm

Here, we will prove the tractability of the $\text{MAP}(\mathcal{LR})$ problem in Theorem 4.5 by construction, so by explicitly constructing the algorithm MPMAP that calculates the argmax exactly and in polynomial time.

To simplify the derivation, we will w.l.o.g. focus on the problem in which the graph $\mathcal{G}$ forms a tree instead of a forest. The approach can be easily extended to forests by looping over the trees.

The key idea behind the message-passing algorithm is that we can exploit the tree structure of the global graph $\mathcal{G}$ (Def. 4.4). As both density and constraints must be compatible, so $\mathcal{G}$ must have a treewidth of 1, we can decompose the computation of the overall maximum into smaller and smaller problems. In order to derive the message-passing, we will again focus on calculating the maximum over the constraints in Ex. 4.1 of the density in Ex. 4.3:

$$\begin{aligned}
 & \max_{\mathbf{x}} (F_3(x_3) F_{21}(x_2, x_1) F_2(x_2)) (F_{31}(x_3, x_1) F_3(x_3)) \\
 &= \max_{x_1} \frac{9}{100} \prod_{i \in \{2,3\}} \underbrace{\max_{x_i} F_{i1}(x_i, x_1) \underbrace{F_i(x_i)}_{=m_{F_{X_i} \rightarrow F_{i1}}(x_1)}}_{=m_{F_{i1} \rightarrow X_1}(x_1)} \\
 &= \max_{x_1} \frac{9}{100} \prod_{i \in \{2,3\}} m_{F_{i1} \rightarrow X_1}(x_1)
 \end{aligned}$$

So, if we manage to “maximize out” (akin to marginalizing out) the variables one by one, e.g., here calculate $\max_{x_i} F_{i1}(x_i, x_1) F_i(x_i)$ as a function of x_1 , we have a scalable algorithm to exactly compute MAP even for high-dimensional problems. We start from a root node X_r , which can be chosen so as to minimize the number of computations, and then recurse into the children of the directed graph until we hit the leaves:

$$m_{X_i \rightarrow F_S}(x_i) := \prod_{c \in \text{child}(X_i)} m_{F_{ci} \rightarrow X_i}(x_i) \cdot F_i(x_i) \quad (8)$$

$$m_{F_{ij} \rightarrow X_j}(x_j) := \max_{x_i} F_{ij}(x_i, x_j) \cdot m_{X_i \rightarrow F_{ij}}(x_i) \quad (9)$$

$$\max_{\mathbf{x}} \prod_S F_S(\mathbf{x}_S) = \max_{x_r} F_r(x_r) \cdot \underbrace{\prod_{c \in \text{child}(X_r)} m_{F_{cr} \rightarrow X_r}(x_r)}_{\text{univariate function}} \quad (10)$$

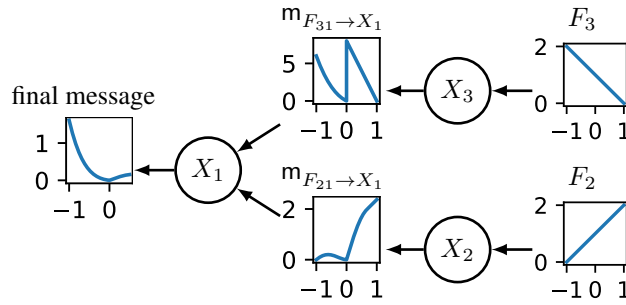
In order to compute the overall maximum, we now only need to maximize the resulting univariate function (line 10). As we only have to maximize out one variable after another, and an operation on single variables, we have a promising direction for a scalable algorithm. It is good to reflect at this point on how the operations used in the definitions of the messages (lines 8 and 9) fit our tractable MAP conditions (TMC, see Def. 4.2). We will first look at (8). Here, we have the product over the incoming messages and the factor for variable x_i . We can reorder this such that we have a product over the SMT formulas (leading to a logical “ $\wedge$ ”) and a product over the functions in Ω . As we are closed under product (and tractable), this message results in a product between the indicator function over an SMT formula and a function in Ω . Then, in (9) we have the product of a factor and the incoming message. We can again reorder them and group them by a product over constraints (which results in a single indicator over the conjunction of the formulas) and over functions (which stays again in Ω). As we later detail in Appendix A.3, we reduce the maximum over the SMT constraints to multiple symbolic maximums with linear upper and lower bounds and pointwise max-comparisons. Both are required to be tractable and stay in our function-class Ω . Finally, in line 10, we have a single maximum, which is just a special case of the symbolic bounds and therefore also tractable (we consider the image of the linear bounds to be the extended reals, so this is the special case of constant upper/lower bounds). We conclude that most of the operations to compute single messages are guaranteed to be tractable directly following from the TMC conditions, except for the enumeration of the linear bounds in Alg. 3. We will later see in the analysis of MPMAP (Appendix A.5) that these are at most polynomially many in the number of atoms per formula.

Focusing back on these messages, we see that there is a remaining piece of the puzzle missing. We do not only need to track the max. value when maximizing out variable after variable, but also the corresponding position this maximum is attained at for the variables maximized out. In comparison to the piecewise message corresponding to the value, which is always a univariate function, this function actually has the signature $\mathbb{R} \rightarrow \mathbb{R}^m$ with m corresponding to the amount of variables maximized out. Therefore, the dimensionality of the space of its image grows during the message-passing. E.g.,

after obtaining our final message in Ex. 4.6 on the left, we can compute the overall argmax by maximizing the remaining message, and computing the overall position by evaluating this function at the position where the maximum of the remaining message is attained.

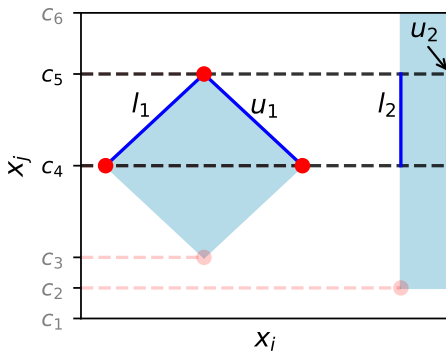
We are now ready to formalize our message-passing scheme in pseudo-code. The high-level routine with the calls to the computation of the messages is given in Alg. 1. As we start with an undirected, but tree-shaped global graph $\mathcal{G}$, we first have to root the tree (or forest). After rooting the tree, the algorithm consists of walking the computed node order and computing the messages over the global graph from bottom to top. Computing these messages is composed of two different operations: one, which we call gather-msgs and is detailed in Alg. 2, collects all the incoming messages at X_i for F_{ij} and multiplies them; the other is called compute-msgs, detailed in Alg. 3, and computes the message from F_{ij} to X_j by maximizing out X_i . The messages at the roots correspond then to the density over X_{root} with all the other variables maximized out, and additionally, as we also track the position of the variables maximized out, a function of X_{root} which returns the assignments of these variables maximized out. Doing one final maximization gives us the MAP-prediction we are looking for.

Example A.1 (MPMAP in action). We show the computed messages for the formula of Ex. 4.1 combined with the density in Ex. 4.3.



Running Alg. 1 on this problem could result in X_1 being picked as root with the order X_3, X_2 , and then X_1 . First, we would call $\text{gather-msgs}(X_3, F_{3,1}) = m_{X_3 \rightarrow F_{3,1}} = F_3$ and then compute $\text{compute-msgs}(F_{3,1}, X_1) = m_{F_{3,1} \rightarrow X_1} = \max_{X_3} F_{3,1}(X_1, X_3) \cdot m_{X_3 \rightarrow F_{3,1}}(X_3)$. The compute-msgs function, in Alg. 3, first gathers the change points for X_j induced by the formula Δ_{ij} via critical-points (see Zeng & den Broeck (2019) for more detail). Inside an interval induced by the change points, so $I = [c, c']$ with c and c' being consecutive change points, we can enumerate the feasible pieces that fall onto interval I . As literals cannot intersect in such an interval (as it would generate another change point), these pieces are defined by single linear upper and lower bounds. So, more formal: $(X_i, X_j) \models \Delta_{piece} \wedge X_j \in I \iff l(X_j) \leq X_i \leq u(X_j) \wedge X_j \in I$ with some linear bound l and u . We visualize this process in Ex. A.2.

Example A.2. We visualize how the critical points lead to a computation of symbolic maximums with linear upper and lower bounds:



On the left, we can see a visualization of how we compute $\max_{x_i} (F_{ij} \cdot m_{X_i \rightarrow F_{ij}})(x_i, x_j)$ via the change points, themselves computed via the intersections (red points). This formula generates the change point $(c_1, c_2, c_3, \dots, c_6)$ for X_j . We iterate over the induced intervals and now visualize the pieces that fall into the interval $I = [c_4, c_5]$.

For $x_j \in I$, we have two blue pieces that fall into the interval. Here, one can clearly see that, on this interval, these pieces define a single linear upper and lower bound for x_i as a function of x_j . These linear bounds are denoted (l_1, u_1) and (l_2, u_2) in the figure.

We can therefore decomposes the overall maximum in this interval first into the maximum over $p_{ij} \cdot m_{X_i \rightarrow F_{ij}}$ over $x_i \in [l_1(x_j), u_1(x_j)]$ and $x_i \in [l_2(x_j), u_2(x_j)]$. This results in another function in Ω and is tractable according to the TMC. To attain the overall maximum with x_j “maximized out”, we have to then do a pointwise max. between the results.

We maximize out X_3 over the pieces (line 6) to obtain the message bound for X_1 . We repeat the same for X_2 . Finally, we call $\text{gather-root}(X_1) = \prod_{i \in \{2,3\}} m_{F_{i1} \rightarrow X_1} = m$ and call the max and argmax over m to obtain both the maximal value and the corresponding coordinates.

While Alg. 3 provides a generic algorithm that works with any family for which the TMC hold, we provide a specific version of the algorithm in Alg. 5 that shows how we handle densities in Ω^{PP} . Here, we do not only enumerate the pieces resulting from Δ_{ij} but also the pieces from the piecewise function p_{ij} , in order to perform our $\max_{x_i} p(x_j)p(x_i) = p(x_j) \max_{x_i} p(x_i)$ -trick piecewise. The algorithm for Ω^{PP} is analogous, except that we go into log-space before calling max-out in order to reuse our polynomial maximization scheme.

The complexity is discussed in Appendix A.5

Algorithm 5 compute-msgs $^{\Omega^{\text{PP}}}(F_{i,j}, X_j)$

input $F_{i,j}$: factor, X_j : variable
output $m_{F_{i,j} \rightarrow X_j}$: message

- 1: $m_{F_{i,j} \rightarrow X_j} \leftarrow \text{empty-piecewise}()$
- 2: $p_{ij} \leftarrow \text{get-weight}(\{ij\}, p)$ {bivariate function associated to $\{ij\}$ }
- 3: $\varphi \leftarrow \bigvee_{\varphi_k \in \text{formulas}(p_{ij})} \varphi_k$
- 4: $\Delta'_{ij} \leftarrow \Delta_{ij} \wedge \varphi$
- 5: $\mathcal{P} \leftarrow \text{critical-points}(\text{overall-bounds}(m_{X_i \rightarrow F_{ij}}), \Delta'_{ij})$
- 6: $\mathcal{I} \leftarrow \text{intervals-from-points}(\mathcal{P})$
- 7: **for** interval $I \in \mathcal{I}$ consistent with formula Δ'_{ij} **do**
- 8: $\mathcal{Q} \leftarrow \text{get-msg-pieces}(m_{X_j \rightarrow F_{ij}}, I, p_{ij})$
 {enumerates every piece that falls into $X_i \in I$ }
- 9: $q'_h \leftarrow q_h^i \cdot \text{max-out}(q_h^j, l_h, u_h) \forall (l_h, u_h, q_h^i, q_h^j) \in \mathcal{Q}$
- 10: $m_{F_{i,j} \rightarrow X_j} \mid_{x_i \in I} \leftarrow \text{max-pieces}(\{q'_h, 0 \leq h < |\mathcal{Q}|\})$
 {The maximum over the pieces forms $m_{F_{i,j} \rightarrow X_j}$ at $x_i \in I$ }
- 11: **end for**
- 12: **return** $m_{F_{i,j} \rightarrow X_j}$

A.3. The max-out Algorithm

While a lot of similarities between computing the argmax and the integral exist, a major difference is that the fundamental operation is not similarly established. As the core operation is not integrating out a variable like $\int_{l(x_j)}^{u(x_j)} q(x_j, x_i) dx_i$, but maximizing out a variable like $\max_{x_i \in [l(x_j), u(x_j)]} q(x_j, x_i)$, we call the corresponding algorithm max-out. It is important to note that l and u are symbolic bounds, so affine functions in x_j and therefore the result of maxing out x_i is a function of x_j . Before introducing our general algorithm for the maximization, we have to detail the function class we are actually able to handle. As we previously described, we are looking two main families of functions, piecewise polynomials Ω^{PP} and piecewise exponentiation polynomials Ω^{PEP} . As we can reduce the task of maximizing exponentiated polynomials to the task of maximizing polynomials via going into log-space (see Alg. 9), the rest of the section detailing our max-out algorithm will therefore only consider the task of maximizing piecewise-polynomials (max-out $^{\text{PP}}$). We can therefore concentrate on deriving an algorithm to maximize a univariate, piecewise polynomial under symbolic linear upper and lower bounds. As the result will again be a piecewise polynomial, we can recursively apply the algorithm over and over in the message-passing. We will first focus on computing the *value* of the polynomial with a variable maxed out, and then later generalize this to also include the coordinate of the variable that got maxed out.

Maximizing a univariate piecewise polynomial. In order to derive our algorithm to explicitly construct $m(y) = \max_{x \in [l(y), u(y)]} q(x)$ for $q(x)$ being a piecewise, but not necessarily continuous, polynomial, we first start by assuming it pointwise in y . We begin with a slight generalization of the extreme-value theorem/interior extremum theorem to piecewise polynomials. Therefore, the general formula to compute the maximum of q over $[l(y), u(y)] := I(y)$ is:

$$\max_{x \in I(y)} q(x) = \max \begin{cases} \max\{q(l(y)), q(u(y))\}, & (a) \\ \max\{q(e) \mid e \in E_q \cap I(y)\}, & (b) \\ \max\{q^*(b) \mid b \in B_q \cap I(y)\} & (c) \end{cases}$$

with E_q denoting the extreme points, B_q denoting the boundaries of the pieces of the piecewise polynomial q . Finally, $q^*(b)$ is defined as $q^*(b) := \max\{\lim_{x \rightarrow b^-} q(x), \lim_{x \rightarrow b^+} q(x)\}$ if both the left and right limits are defined, otherwise it

is $q^*(b) := q(b)$. As we can see, we have two piecewise constant terms, the maxima over the extreme (b) and boundary-points (c), and only one term, $\max\{q(l(y)), q(u(y))\}$, being directly dependent on y . But when can terms (b) and (c) change their values? We have to look at the union of the feasible pre-images of these points $C_{\hat{e}} = \bigcup_{f \in l, u} \{y \mid f(y) \in E_q \cup B_q \wedge l(y) \leq u(y) \wedge f \neq \text{const.}\}$. So all the valid points ($l(y) \leq u(y)$) that map to extreme and boundary points of q . Inside an interval spanned by these points, no change of (b) and (c) can occur as no point from E_q and B_q can enter $I(y)$, and therefore the terms (b) and (c) are constants. We call this set $C_{\hat{e}}$, which forms our first contribution to the set of breakpoints of the resulting piecewise function $m(y)$. The other term (a) is more tricky to analyze. As we want to explicitly construct our resulting piecewise function $m(y)$, the first question to ask is where the breakpoints originating from the term $\max\{q(l(y)), q(u(y))\}$ are located. A first contribution to the breakpoints, which we call C_{switch} , comes from change in the dominating polynomial, which must occur at the feasible roots of $q(l(y)) - q(u(y))$, so roots of $t(y) = q(l(y)) - q(u(y))$ with $l(r) \leq u(r)$. As we are only interested in the real roots here, we can run a real root isolation algorithm like the Vincent-Akritas-Strzeboński continued-fraction method (Vincent, 1834; Akritas & Strzebonski, 2005). So, inside the intervals spanned by the breakpoints $C_{\hat{e}} \cup C_{\text{switch}}$ we know that terms (b) and (c) are constants, which we can calculate and henceforth call $\hat{e}$, and that only one polynomial must dominate. We denote the dominating polynomial on interval I as $\hat{q}$, so:

$$\hat{q} = (q \circ u) \iff \forall y \in I : (q \circ u) \geq (q \circ l) \quad (11)$$

$$\hat{q} = (q \circ l) \text{ otherwise} \quad (12)$$

We now only need to characterize the relationship between $\hat{q}$ and $\hat{e}$. The careful reader will notice that $C_{\hat{e}}$ also characterizes the behavior of $\hat{q}$ between its breakpoints, as it lets us assume monotonicity for $\hat{q}$ inside the intervals, because it is composed of the extreme points of both $q \circ l$ and $q \circ u$. This observation enables us to enumerate all the possible relationships between $\hat{e}$ and $\hat{q}$ inside an interval $[i_1, i_2] = I'$, with $[i_1, i_2]$ being an interval spanned by the breakpoints $C_{\hat{e}} \cup C_{\text{switch}}$, them being:

1. $\min\{\hat{q}(i_1), \hat{q}(i_2)\} \geq \hat{e} \Rightarrow \forall i \in I' : \max\{\hat{q}(i), \hat{e}\} = \hat{q}(i)$
2. $\max\{\hat{q}(i_1), \hat{q}(i_2)\} \leq \hat{e} \Rightarrow \forall i \in I' : \max\{\hat{q}(i), \hat{e}\} = \hat{e}$
3. $\hat{q}(i_1) < \hat{e} < \hat{q}(i_2)$, then there exists $i_{\text{break}} \in I'$ s.t.

$\forall i \in [i_1, i_{\text{break}}] :$	$\max\{\hat{q}(i), \hat{e}\} = \hat{e},$
$\forall i \in [i_{\text{break}}, i_2] :$	$\max\{\hat{q}(i), \hat{e}\} = \hat{q}(i)$
4. $\hat{q}(i_1) > \hat{e} > \hat{q}(i_2)$, then there exists a $i_{\text{break}} \in I'$ s.t.

$\forall i \in [i_1, i_{\text{break}}] :$	$\max\{\hat{q}(i), \hat{e}\} = \hat{q}(i),$
$\forall i \in [i_{\text{break}}, i_2] :$	$\max\{\hat{q}(i), \hat{e}\} = \hat{e}$

In order to simplify the math, we will set $\hat{q}(y) = -\infty$ if y is not in the domain of $\hat{q}$.

But, a final contribution to the set of breakpoints of m is still missing. We need to add the start/end-bounds contributed by u and l , as the start/end of our feasible area $l(y) \leq u(y)$, if it exists, contributes another breakpoint. As two linear functions can only intersect at most once, and this intersection can be either the start or end-point of our feasible area, the possible values of the set C_{bounds} are straightforward to enumerate:

$$C_{\text{bounds}} = \begin{cases} \{y_s, +\infty\}, & \text{if } \exists y_s : l(y_s) = u(y_s) \wedge y_s \text{ is start} \\ \{-\infty, y_s\}, & \text{if } \exists y_s : l(y_s) = u(y_s) \wedge y_s \text{ is end} \\ \emptyset, & \text{if } \forall y : u(y) < l(y) \quad (\text{parallel}) \\ \{-\infty, +\infty\}, & \text{otherwise.} \end{cases}$$

Together, these sets form the breakpoints of our piecewise function $m(y)$: $C_{\text{break}} = C_{\hat{e}} \cup C_{\text{switch}} \cup C_{\text{bounds}}$. We are now ready to formalize our max-out^{PP}-algorithm, as provided in Alg. 6.

Algorithm 6 max-out^{PP}(q, l, u)

input q : piecewise polynomial, l : affine lower bound, u affine upper bound

output m : piecewise polynomial

```

1:  $V_q, C_{\hat{e}}, C_{switch}, C_{bounds} \leftarrow \text{prepare-breaks}(q, l, u)$ 
2:  $m \leftarrow \text{empty-piecewise}()$ 
3: for  $[i_1, i_2] \in \text{intervals}(C_{\hat{e}} \cup C_{switch} \cup C_{bounds})$  do
4:    $\hat{q} \leftarrow \text{dominating-poly}((q \circ l), (q \circ u), i_1, i_2)$ 
5:    $\hat{e} \leftarrow \text{inner-max}(q, l, u, i_1, i_2, V_q)$ 
6:    $\$ \leftarrow \text{exclusive-or-inclusive-start}(\hat{q})$ 
7:    $\{\$ \text{ can be either "[]" or "()" }\}$ 
8:   if  $\max\{\hat{q}(i_1), \hat{q}(i_2)\} \leq \hat{e}$  then
9:     if  $\hat{e} = -\infty$  then
10:      continue
11:     else
12:        $m|_{\$[i_1, i_2]} \leftarrow \hat{e}$  {defines  $m$  on interval  $\$[i_1, i_2]$ }
13:     end if
14:   else if  $\min\{\hat{q}(i_1), \hat{q}(i_2)\} \geq \hat{e}$  then
15:      $m|_{\$[i_1, i_2]} \leftarrow \hat{q}$ 
16:   else if  $\hat{q}(i_1) < \hat{e} < \hat{q}(i_2)$  then
17:      $i_{break} \leftarrow \text{intersection}(\hat{q}, \hat{e}, i_1, i_2)$ 
18:      $m|_{\$[i_1, i_{break}]} \leftarrow \hat{e}$ 
19:      $m|_{\$[i_{break}, i_2]} \leftarrow \hat{q}$ 
20:   else
21:      $i_{break} \leftarrow \text{intersection}(\hat{q}, \hat{e}, i_1, i_2)$ 
22:      $m|_{\$[i_1, i_{break}]} \leftarrow \hat{q}$ 
23:      $m|_{\$[i_{break}, i_2]} \leftarrow \hat{e}$ 
24:   end if
25: end for
26: if  $C_{bounds}$  has a finite element then
27:    $i_b \leftarrow \text{get-finite-elem}(C_{bounds})$ 
28:    $m|_{[i_b, i_b]} \leftarrow (q \circ l)(i_b)$  {start or end}
29: end if
30: return  $\text{simplify}(m)$ 

```

A.4. Correctness of the max-out Algorithm

Before proving the correctness of the max-out^{PP}-algorithm, we will need a few propositions later used in the proof of Theorem A.5.

Consider a univariate piecewise polynomial q with finitely many (not necessarily consecutive) pieces with discontinuous points at the breakpoints, and let $C_{\hat{e}}, C_{switch}, C_{bounds}, V_q$ be defined as in Alg. 6. Let l and u be univariate affine functions, and define $I(y) := [l(y), u(y)]$. We further define $q'(x)$ to be the maximum between the left and right hand limit ($\lim_{x' \rightarrow x-} q(x)$ and $\lim_{x' \rightarrow x+} q(x)$), if they exist, otherwise the existing left or right. With $\text{int}(A)$ we denote the interior of a set A .

We will treat $\max \emptyset$ and $\sup \emptyset$ as undefined.

Proposition A.3. *For any y with $l(y) < u(y)$, we have*

$$\begin{aligned} \sup_{x \in I(y)} q(x) = \max \Big(& \{\max(q(u(y)), \lim_{x \rightarrow u(y)-} q(x)) \mid u(y) \in \text{dom}(q)\}, \\ & \{\max(q(l(y)), \lim_{x \rightarrow l(y)+} q(x)) \mid l(y) \in \text{dom}(q)\}, \\ & \{q'(e) \mid e \in (E_q \cup B_q) \cap \text{int}(I(y))\} \Big). \end{aligned} \quad (13)$$

Proof. The restriction of q to $I(y)$ decomposes into finitely many continuous polynomial pieces. On each (closed relative to $I(y)$) piece, the continuous polynomial attains its maximum either at an interior critical point or at an endpoint of that piece. Interior critical points lie in $E_q \cap \text{int}(I(y))$; endpoints are either interior breakpoints (via q' over $E_q \cap \text{int}(I(y))$) or the interval endpoints $l(y), u(y)$. Any approach to an endpoint from inside $I(y)$ produces only the interior-directed one-sided limit ($\lim_{x \rightarrow u(y)-} q(x)$ at $u(y)$, $\lim_{x \rightarrow l(y)+} q(x)$ at $l(y)$), or the point value, over which we take the maximum. Taking the maximum over this finite candidate set yields the stated equality. $\square$

Proposition A.4. *For any y contained in an interval $[i_1, i_2]$ spanned by the breakpoints $C_{\hat{e}} \cup C_{switch} \cup C_{bounds}$, we have*

$$\{E_q \cup B_q\} \cap (\min\{l(i_1), l(i_2)\}, \max\{l(i_1), l(i_2)\}) \setminus S = \emptyset \quad (14)$$

and

$$\{E_q \cup B_q\} \cap (\min\{u(i_1), u(i_2)\}, \max\{u(i_1), u(i_2)\}) \setminus S = \emptyset \quad (15)$$

with $S = \{E_q \cup B_q\} \cap [\max\{l(i_1), l(i_2)\}, \min\{u(i_1), u(i_2)\}]$

Proof. Since l and u are affine, they are either strictly monotone or constant on $[i_1, i_2]$.

If l (resp. u) is strictly monotone, then it defines a bijection between $[i_1, i_2]$ and $[l(i_1), l(i_2)]$ (resp. $[u(i_1), u(i_2)]$), and thus any break-point or extreme point of q in this image would correspond to one of the known points in $C_{\hat{e}} \cup C_{switch}$. If l (resp. u) is constant, then its image is a singleton possibly coinciding with a break-point or extremum of q , in which case the point is already contained in S . $\square$

With the above propositions, we are now ready to prove correctness.

Theorem A.5 (max-out^{PP} is correct). *For any piecewise polynomials q , which may contain non-continuous breakpoints, for all affine functions l, u and any real number y , we have that $\sup_{x \in [l(y), u(y)]} q(x) = \text{max-out}^{PP}(q, l, u)(y)$.*

Ansatz. For the proof of correctness we fill proof that for some arbitrary piecewise polynomial q , upper bound l , lower bounds u and point y , if $\sup_{x \in [l(y), u(y)]} q(x) = z$, then $\text{max-out}^{PP}(q, l, u)(y) = z$. Since $\sup_{x \in [l(y), u(y)]} q(x)$ is a total function in y with the same domain as $\text{max-out}^{PP}(q, l, u)$, their equality implies that the inverse relation also holds, establishing correctness. We do this proof over $\text{max-out}^{PP}(q, l, u)(y)$ with the simplify-call omitted, as it only joins the pieces of the same consecutive polynomial it results in the same function point-wise.

Proof. **We start the proof by considering the case of $l(y) > u(y)$.** As in this case $\sup_{x \in [l(y), u(y)]} q(x) = \max \emptyset$ and we treat $\max \emptyset$ as undefined, we start to the proof by checking that $\text{max-out}^{PP}(q, l, u) =: m$ is undefined whenever $l(y) > u(y)$. There are essentially four cases to check here, as l and u are linear functions, they can only intersect at most once:

1. if $[l(y), u(y)]$ is feasible for all y (no intersection of l and u), then there is nothing to check as $\max \emptyset$ never occurs
2. if $[l(y), u(y)]$ is infeasible for all y (no intersection of l and u), then $C_{\hat{e}} \cup C_{\text{switch}} \cup C_{\text{bounds}} = \emptyset$, as we $C_{\hat{e}}$ and C_{switch} only contain feasible points and C_{bounds} is also defined as the empty set for this case. As the domain of m is defined by the intervals spanned by these points, it is also empty
3. if $[l(y), u(y)]$ is bounded from below by y_{start} (intersection at y_{start}), then $\min C_{\hat{e}} \cup C_{\text{switch}} \geq y_{\text{start}}$ and $\min C_{\text{bounds}} = y_{\text{start}}$, therefore m is undefined for $y < y_{\text{start}}$
4. if $[l(y), u(y)]$ is bounded from above by y_{end} (intersection at y_{end}), then $\max C_{\hat{e}} \cup C_{\text{switch}} \leq y_{\text{end}}$ and $\max C_{\text{bounds}} = y_{\text{end}}$, therefore m is undefined for $y > y_{\text{end}}$

This settles the case. $\checkmark$

We will now check the case for $l(y) = u(y)$. Let y be some real number such that $l(y) = u(y)$. In this case we have $\sup_{x \in I(y)} q(x) = (q \circ u) = (q \circ l) = z$. As l and u are linear functions, they can only intersect once: at the start or the end of the feasible set (in terms of y). Therefore, in case $l(y) = u(y)$, then y is the only finite element in the set C_{bounds} . This element is called i_b in the max-out^{PP} algorithm, for which we set m to be $(q \circ l)(i_b)$, which is the value of $\sup_{x \in I(y)} q(x)$. $\checkmark$

We can now assume that $l(y) < u(y)$. We can therefore turn our attention to Proposition A.3:

$$\sup_{x \in I(y)} q(x) = \max \left(\begin{aligned} &\{\max(q(u(y)), \lim_{x \rightarrow u(y)^-} q(x)) \mid u(y) \in \text{dom}(q)\}, \\ &\{\max(q(l(y)), \lim_{x \rightarrow l(y)^+} q(x)) \mid l(y) \in \text{dom}(q)\}, \\ &\{q'(e) \mid e \in (E_q \cup B_q) \cap \text{int}(I(y))\} \end{aligned} \right) \quad (16)$$

This expression is composed of two groups, the (1) endpoints (first two line) and (2) interior points (the last line). We will continue this proof by case distinction on these two groups.

Case I: q is undefined on $I(y) = [l(y), u(y)]$. Before considering the maximum to be either from set (1) or (2), we have to check an additional case: if both sets are empty because q is undefined on $I(y)$, then $\sup_{x \in I(y)} q(x)$ is undefined as well. We therefore have to check that $m := (\text{max-out}^{\text{PP}}(q, l, u))$ is also undefined in this instance. As $\min C_{\text{bounds}} \leq y \leq \max C_{\text{bounds}}$, we are guaranteed to find an $[i_1, i_2] \ni y$ in the set of breakpoints. As q is entirely undefined on $I(y)$, both $q \circ l$ and $q \circ u$ have to be undefined on $[i_1, i_2]$, as no breakpoints for the respective functions can lie in the interval. As a consequence, dominating-poly($(q \circ l), (q \circ u), i_1, i_2$) has to return $y \mapsto -\infty (= \hat{q})$. As $V_q \cap I(y) = \emptyset$, by assumption, it follows that $V_q \cap I(y) \supseteq V_q \cap [\max\{l(i_1), l(i_2)\}, \min\{u(i_1), u(i_2)\}] = \emptyset$. Therefore, $\hat{e}$ in line 2 (Alg. 12) takes the value $-\infty$. Therefore:

$$\max\{\hat{q}(i_1), \hat{q}(i_2)\} \leq \hat{e} \quad (17)$$

$$\Rightarrow \max\{-\infty, -\infty\} \leq -\infty \quad (18)$$

$$\Rightarrow -\infty \leq -\infty \quad (19)$$

As $\hat{e} = -\infty$, we do not assign m any value in this interval (following from line 10), and therefore m is undefined. $\checkmark$

As we can now assume that q is at least partially defined on $I(y)$, **we know that either set (1) or set (2) must be non-empty.** We now have to prove that for either cases, max-out^{PP} returns the correct result.

Case II: The maximum is in set (1) (so we assume the supremum occurs at the boundary). Let $\hat{q}$ be either $q \circ l$ or $q \circ u$ depending on which set the supremum came from (or an arbitrary choice for a tie). Let w.l.o.g. the upper bound be the winning bound, we therefore have to analyze the expression:

$$\max(q(u(y)), \lim_{x \rightarrow u(y)^-} q(x)) \quad (20)$$

We will now have another case distinction on the type of $u(y)$ for q under the assumption that the supremum came from $q \circ u$.

Case II.I: Assuming $u(y)$ is not a break point of m , the limit coming from the interior will coincide with the point-wise evaluation $q(u(y)) = \hat{q}(y)$. Let $[i_1, i_2]$ be the currently active breakpoints for y . If it is a tie, the chosen polynomial returned by dominating-poly (Alg. 10) does not matter. If $q(u(y))$ dominates $q(l(y))$, as we assume, it must do so on the whole interval, therefore on i_1 , $q(u(y')) - q(l(y'))|_{y'=i_1}$ must be positive or in case it is zero, the highest non-vanishing gradient of $q(u(y')) - q(l(y'))$ must be positive for $q(u(y')) - q(l(y'))|_{y'=y} > 0$ to hold. This follows from Taylor's theorem with remainder (Apostol, 1991) around i_1 and generalized to the interval using the fact that the two polynomials can not intersect inside the interval. Therefore, dominating-poly (Alg. 10) in all cases returns $q \circ u$ and it is assigned as $\hat{q}$. We need to collect all the breakpoints/extreme points inside $[l(y), u(y)]$, but since we know that between i_1 and i_2 there are no breakpoints/extreme points, $[\max\{l(i_1), l(i_2), \min\{u(i_1), u(i_2)\}\}, \min\{u(i_1), u(i_2)\}\}] \subseteq [l(y), u(y)]$ contains all the relevant points and does not change for $y \in (i_1, i_2)$. In case the set is non-empty, max over (2) is the same as $\hat{e}$ (line 2, Alg. 12). As $\hat{e}$ and $\hat{q}$ can not intersect inside the interval, as we also split by intersection (if intersection occurs, and there can only be one, cases line 16 and 20 handle them by splitting the interval), therefore we know that $\hat{e} \leq \hat{q}(i_1)$ and $\hat{e} \leq \hat{q}(i_2)$ following from $\hat{e} \leq \hat{q}(y)$. So, we can either be in case line 15, 19 or 22. In conclusion, $m(y)$ is assigned $\hat{q}$ and $m(y) = \hat{q}(y) = q(u(y)) = \sup_{x \in [l(y), u(y)]} q(x)$. ✓

Case II.II: Assuming $u(y)$ is a break point of m , there might be a difference between $q(u(y))$ and $\lim_{x \rightarrow u(y)-} q(x)$. A first observation is that, in case we look at a breakpoint where a polynomial starts closed (and the other polynomial ends open), we have $\lim_{x \rightarrow u(y)-} q(x)$ to be in the values picked up by $\hat{e}$, as here we have an inclusive comparison (line 2 Alg. 12). In this case, the point-wise $q(u(y))$ can be strictly greater than $\lim_{x \rightarrow u(y)-} q(x)$ or otherwise. We therefore have two cases:

We will additionally assume $\lim_{x \rightarrow u(y)-} q(x) < q(u(y))$. As this can only happen if we look at a break-point that ends open and starts closed, in order for $q(u(y))$ to be extracted by dominating-poly, we have to check that the break-point is correctly assigned to the right interval. As we keep track of the open/closeness properties at the start of the interval (line 6) and assign it correctly, potentially overriding the previous interval, as every assignment starts with \$. We therefore look at the correct segment, so m starts with a segment at y that starts closed. Therefore $q(u(y))$ is extracted by dominating-poly. As $q(u(y)) \geq (2)$ if set (2) is not empty, and $q(u(y)) > \lim_{x \rightarrow u(y)-} q(x)$, we have $q(u(y)) = \hat{q}(i_1) \geq \hat{e}$, where $i_1 \in C_{\hat{e}}$. We can therefore enumerate all the possibilities for i_2 and check whether m returns the correct value for this: In case $\hat{q}(i_2) \geq \hat{e}$, we have line 15 and $m(i_1) = m(y) = \hat{q}(y)$ (as "\$ = [")". In case $\hat{q}(i_2) < \hat{e}$, we can either have line 19 in case $\hat{q}(i_1) = \hat{e}$, then $m(i_1) = m(y) = \hat{e} = \hat{q}(y)$. Or we have line 22 and again $m(i_1) = m(y) = \hat{q}(y)$ (as "\$ = [")". ✓

We will now assume $\lim_{x \rightarrow u(y)-} q(x) \geq q(u(y))$. It is important to note that in case the break-point starts open/ends closed, we actually have $\lim_{x \rightarrow u(y)-} q(x) = q(u(y))$. So let us quickly consider this case, so when y is the end of the segment ($y = i_2$). So from our assumptions of the maximum being in set (1), we know that $\hat{q}(i_2) \geq \hat{e}$. As for all possibilities (line 15 and 19), the assignment is closed to the right, so after running this iteration, we have $m(y) = m(i_2) = \hat{q} = \lim_{x \rightarrow u(y)-}$. In order for the break-point to belong to the previous interval, the next assignment must start open ("\$ = (")", which is respect in all the possible next assignments (so line 12, 15, 18 and 22). Therefore, we have that m at y falls into the previous interval and therefore $m(y) = \lim_{x \rightarrow u(y)-} q(x)$. In the other case, so we start closed and end open ($y = i_1$), the argument is the same as in the $\lim_{x \rightarrow u(y)-} q(x) < q(u(y))$ but with the assumption that $q(u(y)) = \hat{q}(i_1) \leq \hat{e}$ as $\lim_{x \rightarrow u(y)-} q(x)$ falls into $\hat{e}$. ✓

Case III: Assuming the maximum is in set (2) (more detailed: we assume the supremum **strictly** occurs inside the interval either as a left/right limit at a break-point or at an extreme-point). Let i_1, i_2 be the left/right boundaries of the interval of m where y falls into. Let e' be the max over set (2), so $e' = \max\{q'(e) \mid e \in (E_q \cup B_q) \cap \text{int}([l(y), u(y)])\}$ (which therefore must be non-empty).

Furthermore, we will do another case distinction based on whether l, u are non-constant or not:

Case III.I: l and u are non-constant As the breakpoints for m contain the set of breakpoints/extreme-points for $q \circ l$ and $q \circ u$, we know that

$$e' = \max\{q'(e) \mid e \in (E_q \cup B_q) \cap \text{int}([l(y), u(y)])\} \quad (21)$$

$$= \max\{q'(e) \mid e \in (E_q \cup B_q) \cap [l(y'), u(y')]\} \quad (22)$$

$$= \hat{e} \quad (23)$$

for $y' \in (i_1, i_2)$, $l' = \max\{l(i_1), l(i_2)\}$ and $u' = \min\{u(i_1), u(i_2)\}$. Then the last equality is trivially true if y is not at the

boundary of i_1, i_2 (as no points of $E_q \cup B_q$ can lie inside this interval). If y is at the boundary of the interval i_1, i_2 , we have to be careful as we know could have potentially added the limit from outside $[l(y), u(y)]$. Fortunately, all is fine. We know that on $[l(y), u(y)]$, e' is dominant by our assumption of Case III (the maximum is in set (2)). But what if $l' = l(y)$ or $u' = u(y)$, then we would have added, via q' , the limit from outside. So the limit from the right towards our y , which is not inside $[l(y), u(y)]$. This can only lead to a problem if we have a closed end/open start at y , as the right at y does not necessarily coincide with $q(y)$ (it does otherwise). But in this case, the picked up interval i_1, i_2 must be one where y must be on the right boundary, so it is closed to the right (it would not have been picked up for the left boundary). But in this case, we can use our assumption of Case III (max must lie in the $\text{int}(I(y))$) as $\hat{q}(y)$ (point wise) would evaluate to the right-limit and is dominated by the interior. So $e' = \hat{e}$ for both cases. Now let $\hat{q}$ be the dominant polynomial on i_1, i_2 as chosen by dominating-poly, so either $q \circ l$ or $q \circ u$ or the function that maps to minus infinity. As we know that $\hat{q}$ is monotonic, we can either have the case of line 19, in which case $m(y) = \hat{e} = e'$, or $\hat{q}$ has to cross $\hat{e}$ for $\hat{q}(y) < \hat{e}$ to hold (assumption Case III).

Then we either have a growing (case if-statement line 23) or falling (case if-statement 27) $\hat{q}$ on the segment i_1, i_2 . By assumption, we must be in the case with $\hat{q}(y) < \hat{e}$, so one of the bounds is actually the i_{break} and we are in the range covered by line 25 or 30 with $y \neq i_{break}$ by our assumption. If y is inside the interval, we now have $m(y) = \hat{e} = e'$. If y is the boundary i_1 or i_2 , we now have $\hat{q}$ to be, in case it is a breakpoint, either the ending or starting polynomial. This depends on whether the interval starts closed or open. But by our assumption of set (2) dominating set (1), we have that $\hat{e}$ is dominating the ending polynomial (in case we have a closed end) or both (in case we have an open end). So we still fall into either case 19, 25 or 30 and therefore $m(y) = \hat{e} = e'$. ✓

□

A.5. Complexity Results for Ω^{PP} and Ω^{PEP}

The idea behind analyzing the computational complexity behind the message-passing algorithm is to focus on the number of pieces generated during the message-passing algorithm, as the symbolic maximum is tractable, which follows from the tractability of the approximate root-enumeration (Schönhage, 1982) even when the degree of the polynomial is an input (runtime $\tilde{O}(d^3 \log(b))$ for approximation error $e = \frac{1}{2^b}$). As extending the max to the argmax results also in a still tractable complexity (the symbolic argmax piecewise polynomial has as many pieces as the max polynomial but only a degree of at most 1), the theoretical analysis boils down to analyzing the total number of pieces sent.

In order to simplify the theoretical analysis, we will first analyze the number of pieces resulting from computing the symbolic max over a piecewise polynomial and symbolic, affine, upper and lower bound.

Proposition A.6. *Computing the symbolic maximum $m(y) = \max_{x \in [l(y) := a \cdot y + b, u(y) := a' \cdot y + b']} p(x)$ for a polynomial p of degree q with $m \geq 1$ pieces results in at most $8mq + 4m + 4$ pieces.*

Proof. We can focus on the number of breakpoints enumerated by the max-out^{PP} algorithm when applied to the polynomial p . We can focus on the algorithm, especially the prepare-breaks-function. Here we have:

$$|E_q| \leq mq \quad (\text{extreme points of } p) \quad (24)$$

$$|B_q| \leq (m + 1) \quad (\text{breakpoints points of } p) \quad (25)$$

$$|C_{\hat{e}}| \leq 2(mq + (m + 1)) \quad (\text{after } l^{-1}(E_1 \cup B_q) \text{ and } u^{-1}(E_1 \cup B_q)) \quad (26)$$

$$|C_{switch}| \leq (2m)q \quad (\text{we have } 2m \text{ pieces to between } p \circ l \text{ and } p \circ u \text{ with max. } q \text{ roots}) \quad (27)$$

$$|C_{bounds}| \leq 1 \quad (\text{we can have only one intersection between } l \text{ and } u) \quad (28)$$

$$|C_{total}| \leq 2(2(mq + (m + 1)) + 2mq + 1) \quad (\text{at most one additional break between those points max-out}^{PP}) \quad (29)$$

$$= 8mq + 4m + 4 \quad (30)$$

□

In order to derive the complexity, we will take a similar approach to Zeng et al. (2020a). We will start with adapting proposition 22 from Zeng et al. (2020a) to our setting:

Proposition A.7. *Suppose the variables X_i and X_j are connected in the global graph by a factor F_{ij} associated to Δ_{ij} of size c . Then:*

1. the number of pieces in $m_{X_i \rightarrow F_{ij}}$ is bounded by $\sum_S m_S$, with $m_S = |m_{f_S \rightarrow X_i}|$ and $S \in \text{neigh}(X_i)/F_{ij}$.
2. for Ω^{PP} : the number of pieces in $m_{F_{ij} \rightarrow X_j}$ is bounded by $(2c + c^2)(c \cdot (8mq + 4m + 4)c^2 \cdot q)$, with $m = |m_{X_i \rightarrow F_{ij}}|$, $q = q_i + \max_{h \in [1, \dots, m]} (\deg(\text{pieces}(m_{X_i \rightarrow F_{ij}})[h]))$ with q_i being the max. degree of the polynomial factor associated with X_i in the weight-function attached to Δ_{ij} .
3. for Ω^{PEP} : the number of pieces in $m_{F_{ij} \rightarrow X_j}$ is bounded by $(2c + c^2)(c \cdot (8mq' + 4m + 4)c^2 \cdot q')$ for $q' = \max\{q_i, \max_{h \in [1, \dots, m]} (\deg(\text{pieces}(m_{X_i \rightarrow F_{ij}})[h]))\}$,

Proof. Statement 1 is directly taken from proposition 22 from Zeng et al. (2020a). It holds for the intersections between the messages due to the overall product resulting in the number of pieces being at most the sum over the number of individual pieces. ✓

The second statement is more challenging and analyzes the number of pieces returned by the algorithm compute-msgs. A key difference between Zeng et al. (2020a) and our approach is that we compute the symbolic maximum over a piecewise function, whereas Zeng et al. (2020a) integrates a single polynomial at a time. Therefore the critical points $\mathcal{P}$ in Zeng et al. (2020a) is both a function of the size of the formula Δ'_{ij} ($= c$, defined in line 4 in Alg. 5) and the number of pieces m . Whereas we only take into account the global bounds (if existing), so treat $m_{X_i \rightarrow F_{ij}}$ as a single piece made of piecewise functions when constructing $\mathcal{P}$ and later perform the max-out^{PP} over the piecewise polynomial. We therefore have $|\mathcal{P}| \leq (2c + c^2)$ following from proposition 22 from Zeng et al. (2020a) for a single piece message. We turn our attention towards the inner loop in compute-msgs. Inside a critical interval, we can only have c different pieces for the polynomial factor associated with the Δ_{ij} . We therefor have c times at most $(8mq + 4m + 4)$ pieces (following proposition A.6). In order to compute the point-wise max, we first need to compute the number of intersections between the pieces, which are bounded by the sum, so we have at most $c(8mq + 4m + 4)$ pieces. On each piece, we now have at most c polynomials of degree q to compare. As we need to do a pairwise comparison, with each comparison resulting in at most q pieces, we have c^2q pieces resulting from the symbolic point-wise maximum between the polynomials. Putting all the factors together, we arrive at $(2c + c^2)(c \cdot (8mq + 4m + 4)c^2 \cdot q)$ ✓

For weights in Ω^{PEP} , the derivation is analogous to Ω^{PP} except that the degree of the polynomial does not grow as a sum of the two degrees but via the maximum over q_i and $\{\deg(\text{pieces}(m_{X_i \rightarrow F_{ij}})[h]) | h \in [1, \dots, m]\}$. This is due to the product of the exponentiated polynomials resulting in the exponential of the sum of the polynomials, and $\deg(p_i + p_h)$ is $\max\{\deg(p_i), \deg(p_h)\}$. □

It is important to note here that these worst-case bounds are really a worst-case scenario that can play out with significantly less complexity in practice. For example, it assumes that every comparison between two polynomials q_1, q_2 of order $\tilde{q}$ in the point-wise maximum will always explode into q pieces, the maximum number of roots of $q_1 - q_2$, which all have to lie inside the interval we are currently looking at. And every further comparison has to again result in 1 pieces and so on the recursion goes until all comparisons are made, so in total the $\frac{c(c-1)}{2}q \leq c^2 \cdot q$ pieces.

We now construct the adjacency matrix M for the graph G that represents the DAG used to run our message passing. So $M_{f_S, f'_S} = 1$ if we compute $m_{f_S \rightarrow f'_S}$ during message passing. One can easily see that M is nilpotent, as M is the adjacency matrix over a DAG, and since m matrix-powers of an adjacency matrix represent m -step reachability in the DAG. The order of the nilpotent matrix M can therefore only be at most the diameter of the global graph, so the longest path between any two vertices (Zeng et al., 2020a, Prop. 21).

First, we will focus on Ω^{PP} , as later the proof for Ω^{PEP} is analogous. We now introduce a vector $v^{(t)}$ that captures the max. number of pieces in the message per factor at time-step t . Calculating $\sum_{t=0}^d \sum_s v_s^{(t)}$ therefore bounds the number of pieces in the messages in MP-MAP. In order to approximate this, we need the maximum degree of the univariate polynomial, which we will call $q_{\max}$; in particular, for a single bivariate piece $q_1^1(x_1)q_1^2(x_2)$ we compute $q_{\max}$ via $\max(\deg(p_1), \deg(p_2))$. We first focus on the relationship between $v^{(t)}$ and $v^{(t-1)}$. We start by reducing $v^{(1)}$ to $v^{(0)}$:

$$v_j^{(1)} \leq \sum_i M_{ij}(2c + c^2)(c \cdot (8v_i^{(0)}(2q_{\max}) + 4v_i^{(0)} + 4)c^2 \cdot (2q_{\max})) \quad (31)$$

$$= (2c + c^2)c \cdot c^2 \cdot 4 \sum_i M_{ij}(2M_{ij}v_i^{(0)}(2q_{\max}) + M_{ij}v_i^{(0)} + 1) \cdot (2q_{\max}) \quad (32)$$

with c being the maximum size over all Δ_{ij} . We have the outer sum as we combine all the incoming messages using a product, leading to a bound over the number of pieces of the sum of the individual pieces. The degree of the polynomial, at the point of being passed to $\text{max-out}^{\text{PP}}$, is $2q_{\max}$ as we have a polynomial of degree $q_{\max}$ coming from the leaves associated with an univariate formula Δ_i and then we combine it with another polynomial associated to Δ_{ij} , also with at most a degree of $q_{\max}$. So we arrive at a total degree of $2q_{\max}$. This can be generalized:

$$\|v^{(t)}\|_1 \leq (2c + c^2)c \cdot c^2 \cdot 4(2t(n-d)q_{\max}) \|M(2v^{(t-1)}2t(n-d)q_{\max} + v^{(t-1)} + 1)\|_1 \quad (33)$$

We have the total degree $2tq_{\max}$ as we associate a factorizable polynomial Δ_{ij} with at most a degree of $q_{\max}$ per variable, of which we have two. We also have the multiplicative factor of $(n-d)$ due to the multiplication in gather-msgs(of which we can have max. $(n-d)$).

Denote with s the cardinality of our set of factors $\mathcal{F}$, and d the diameter of $\mathcal{G}$. We can now use the following inequalities $\|M\|_1 \leq s$ and $\|v^{(0)}\|_1 \leq cs$ to derive a bound for $\|\sum_{t=0}^d v^{(t)}\|_1$.

Theorem A.8. *For weights in Ω^{PP} , we have an overall bound of $\|\sum_{t=0}^d v^{(t)}\|_1 = \mathcal{O}(c^{5d+1}(n-d)^{2d}q_{\max}^{2d}(2n)^{d+1}(d!)^2)$ for the number of generated messages during the message-passing.*

Proof.

$$\|v^{(t)}\|_1 \leq (2c + c^2)c \cdot c^2 \cdot 4(2t(n-d)q_{\max}) \|M(2v^{(t-1)}2t(n-d)q_{\max} + v^{(t-1)} + 1)\|_1 \quad (34)$$

$$\leq (2c + c^2)c \cdot c^2 \cdot 4(2t(n-d)q_{\max}) (\|M(2v^{(t-1)}2t(n-d)q_{\max} + v^{(t-1)})\|_1 + s) \quad (35)$$

$$\leq (2c + c^2)c \cdot c^2 \cdot 4(2t(n-d)q_{\max}) (s\|(2v^{(t-1)}2t(n-d)q_{\max} + v^{(t-1)})\|_1 + s) \quad (36)$$

$$\leq (2c + c^2)c \cdot c^2 \cdot 4(2t(n-d)q_{\max}) (s\|v^{(t-1)}(4t(n-d)q_{\max} + 1)\|_1 + s) \quad (37)$$

$$\leq (2c + c^2)c \cdot c^2 \cdot 4(2t(n-d)q_{\max}) (4t(n-d)q_{\max} + 1)(s\|v^{(t-1)}\|_1 + s) \quad (38)$$

$$\leq (2c + c^2)c^3 \cdot 8(4t^2(n-d)^2q_{\max}^2 + q_{\max}^2)(s\|v^{(t-1)}\|_1 + s) \quad (39)$$

$$\leq ((2c + c^2)c^3 \cdot 8 \cdot 5(n-d)^2q_{\max}^2s)t^2(\|v^{(t-1)}\|_1 + 1) \quad (40)$$

$$\leq ((2c + c^2)c^3 \cdot 8 \cdot 5(n-d)^2q_{\max}^2s)t^22\|v^{(t-1)}\|_1 \quad (41)$$

$$\leq Kt^2\|v^{(t-1)}\|_1 \quad (42)$$

with $K = (2c + c^2)c^3 \cdot 5 \cdot 8 \cdot 2(n-d)^2q_{\max}^2s$.

Using this recurrence, we focus on the overall number of messages:

$$\|\sum_{t=0}^d v^{(t)}\|_1 \leq \sum_{t=0}^d \|v^{(t)}\|_1 \quad (43)$$

$$\leq \sum_{t=0}^d [Kt^2\|v^{(t-1)}\|_1] \quad (44)$$

$$\leq \sum_{t=0}^d [K^t(t!)^2\|v^{(0)}\|_1] \quad (45)$$

$$\leq \sum_{t=0}^d K^t(t!)^2cs \quad (46)$$

$$\Rightarrow \|\sum_{t=0}^d v^{(t)}\|_1 = \mathcal{O}(csK^d(d!)^2) \quad (\text{final term dominates, see ratio of the terms in the sum}) \quad (47)$$

We also know that $K = \mathcal{O}(c^5 q_{max}^2 s)$, therefore:

$$\left\| \sum_{t=0}^d v^{(t)} \right\|_1 = \mathcal{O}(cs(c^5(n-d)^2 q_{max}^2 s)^d (d!)^2) \quad (48)$$

$$= \mathcal{O}(c^{5d+1}(n-d)^{2d} q_{max}^{2d} s^{d+1} (d!)^2) \quad (49)$$

As we know that $s \leq 2n$, we have $\left\| \sum_{t=0}^d v^{(t)} \right\|_1 = \mathcal{O}(c^{5d+1}(n-d)^{2d} q_{max}^{2d} (2n)^{d+1} (d!)^2)$.

□

After deriving the computational complexity for weights in Ω^{PP} , we will now focus on weights in Ω^{PEP} . The general approach is the same as in Theorem A.8 with an important difference: the bound on the degree of the exponentiated polynomial stays constant and does not increase in depth. This is due to the product of the exponentiated polynomials turning into the exponential of the sum of the polynomials, which does not increase the degree.

Theorem A.9. *For weights in Ω^{PEP} , we have an overall bound of $\left\| \sum_{t=0}^d v^{(t)} \right\|_1 = dc^{5d+1}(2n)^{d+1} q_{max}^{2d}$ for the number of generated messages during the message-passing.*

Proof. We first start with the expression of the recursion

$$\|v^{(t)}\|_1 \leq (2c + c^2)c \cdot c^2 \cdot 4 \cdot q_{max} \|M(2v^{(t-1)}q_{max} + v^{(t-1)} + 1)\|_1 \quad (50)$$

$$\leq (2c + c^2)c \cdot c^2 \cdot 4(q_{max})(s\|(2v^{(t-1)}q_{max} + v^{(t-1)})\|_1 + s) \quad (51)$$

$$\leq (2c + c^2)c \cdot c^2 \cdot 4(q_{max})(s\|v^{(t-1)}(2q_{max} + 1)\|_1 + s) \quad (52)$$

$$\leq (2c + c^2)c \cdot c^2 \cdot 4(q_{max})(2q_{max} + 1)(s\|v^{(t-1)}\|_1 + s) \quad (53)$$

$$\leq (2c + c^2)c \cdot c^2 \cdot 4(2q_{max}^2 + 1)(s\|v^{(t-1)}\|_1 + s) \quad (54)$$

$$\leq (2c + c^2)c \cdot c^2 \cdot 4(2q_{max}^2 + q_{max}^2)(s\|v^{(t-1)}\|_1 + s) \quad (55)$$

$$\leq (2c + c^2)c \cdot c^2 \cdot 4(3q_{max}^2)(s\|v^{(t-1)}\|_1 + s) \quad (56)$$

$$\leq (2c + c^2)c \cdot c^2 \cdot 4(3q_{max}^2)s(\|v^{(t-1)}\|_1 + 1) \quad (57)$$

$$\leq (2c + c^2)c \cdot c^2 \cdot 4(3q_{max}^2)s2\|v^{(t-1)}\|_1 \quad (58)$$

$$\leq (2c + c^2)c \cdot c^2 \cdot 8(3q_{max}^2)s\|v^{(t-1)}\|_1 \quad (59)$$

$$= K'\|v^{(t-1)}\|_1 \quad (60)$$

with $K' = (2c + c^2)c \cdot c^2 \cdot 8(3q_{max}^2)s$.

Using this recurrence, we focus on the overall number of messages:

$$\left\| \sum_{t=0}^d v^{(t)} \right\|_1 \leq \sum_{t=0}^d \|v^{(t)}\|_1 \quad (61)$$

$$\leq \sum_{t=0}^d [K'\|v^{(t-1)}\|_1] \quad (62)$$

$$\leq \sum_{t=0}^d [(K')^t \|v^{(0)}\|_1] \quad (63)$$

$$\leq \sum_{t=0}^d (K')^t cs \quad (64)$$

$$\Rightarrow \left\| \sum_{t=0}^d v^{(t)} \right\|_1 = \mathcal{O}(dcs(K')^d) \quad (65)$$

As also know that $K' = \mathcal{O}(c^5 q_{max}^2 s)$ and $s \leq 2n$, we have:

$$\left\| \sum_{t=0}^d v^{(t)} \right\|_1 = \mathcal{O}(dcs(K')^t) \quad (66)$$

$$= \mathcal{O}(dc2n(c^5 q_{max}^2 2n)^d) \quad (67)$$

$$= \mathcal{O}(dc^{5d+1}(2n)^{d+1}q_{max}^{2d}). \quad (68)$$

□

We will now prove Theorem 4.5, which we will restate first:

Theorem A.10 (Tractability of $\text{MAP}(\mathcal{LR}\mathcal{A})$). *If the global graph of $\text{MAP}(\mathcal{LR}\mathcal{A})$ has treewidth one and bounded diameter, and the density fulfills the TMC (Def. 4.2), then $\text{MAP}(\mathcal{LR}\mathcal{A})$ can be solved tractably.*

Proof. We will first look at the tractability of the algorithms gather-msgs and compute-msgs for arbitrary densities that fulfill the TMC. A first important observation is that while the symbolic supremum is in itself tractable, as required by the TMC, nesting the symbolic supremum n -times is not tractable, as the output size of the symbolic supremum can scale polynomially. So:

$$\sup_{(x_1, x_2, \dots, x_n)} f(\mathbf{x}) = \sup_{x_1 \in [a, b]} f'(x_1, x_2) \left(\sup_{x_x \in [l_1(x_1), u_1(x_2)]} f''(x_2, x_3) \left(\sup_{x_3 \in [l_2(x_2), u_2(x_2)]} f'''(x_3, x_4) \dots \right) \right) \quad (69)$$

is not tractable in n . In comparison, this is not the case with the point-wise product.

As gather-msgs (and gather-root) consists of the pointwise-product operation, repeated calling of these methods is not an issue. In comparison, compute-msgs is more costly. First, we have a loop over the intervals spanned by the critical points (line 3). As we have seen in the proof of Proposition A.7, these are polynomially many in the number of atoms c in Δ_{ij} . Then, we do the symbolic supremum for each interval, which is tractable and therefore has a polynomial output size. We then do the pointwise maximum between the resulting functions (line 7), again tractable but also polynomial output-size. Calling it once is therefore tractable, but the problem is that nesting these calls is not tractable anymore. This is due to the same reason as Eq. (69) is not tractable: the output size is polynomial in input size.

The question, therefore, is on what does the recursion-depth of compute-msgs (so repeated application of compute-msgs to output generated by compute-msgs) depend during the computation of MPMAP ? This becomes obvious once we pivot our attention from the pseudocode to the mathematical definition of the messages in (4), (5), and (6). As we traverse the tree (or multiple trees in case of a forest) recursively from root to children, this number depends on the longest path found in the graph $\mathcal{G}$, also called the diameter of the graph. But as we assume boundedness of the diameter, the length of the path can not be arbitrarily long, even in varying dimensions. An example of this would be the diameter of the *STAR*-problems in the experiments-section for MPMAP 6. Therefore, $\text{MAP}(\mathcal{LR}\mathcal{A})$ is tractable if the global graph of $\text{MAP}(\mathcal{LR}\mathcal{A})$ has treewidth one and bounded diameter, assuming the density fulfills the tractable map-conditions. □

A.6. Additional Routines for MPMAP

Algorithm 7 gather-root(X_i)

input X_i : variable

output q : piecewise polynomial

- 1: $Q \leftarrow \{m_{F_{j',i} \rightarrow X_i} \mid \forall j' \in \text{neigh}(i)\} \cup \{m_{F_i \rightarrow X_i}\}$
 - 2: **return** $\prod_i Q_i$ {point-wise product}
-

Algorithm 8 get-msg-pieces($m_{X_j \rightarrow F_{ij}}, I, p, \Delta'_{ij}$)

input $m_{X_j \rightarrow F_{ij}}$: univariate Ω element, I : tuple start/end, p bivariate Ω element
output $\mathcal{Q}$: list of (lower-bound, upper bound, univariate function, univariate function)
1: $\mathcal{I} \leftarrow \text{find-symbolic-bounds-in}(I, \Delta'_{ij})$ {list of linear inequalities for X_j wrt X_i }
2: $r \leftarrow []$
3: **for** $k \in [1, \text{len}(\mathcal{Q})]$ **do**
4: $q_k^i(x_i) \cdot q_k^j(x_j) \leftarrow \text{piece}(\omega, \mathcal{Q}_k, I)$ {gets the factorized, active piece}
5: $(l_k, u_k) \leftarrow \mathcal{Q}_k$ {Current formula in the form $l_k(X_i) \leq X_j \leq u_k(X_i)$ }
6: append($r, (l_k, u_k, q_k^i(x_i), q_k^j(x_j) \cdot m_{X_j \rightarrow F_{ij}})$)
7: **end for**
8: **return** r

Algorithm 9 max-out(q, l, u)

input q : piecewise polynomial or piecewise exponentiated polynomial, l : affine lower bound, u affine upper bound
output m : piecewise polynomial or piecewise exponentiated polynomial
1: **if** is-piecewise-exponentiated(q) **then**
2: $q' \leftarrow \log q$ {piecewise log}
3: **else**
4: $q' \leftarrow q$
5: **end if**
6: $m \leftarrow \text{max-out}^{\text{PP}}(q', l, u)$
7: **if** is-piecewise-exponentiated(q) **then**
8: $m' \leftarrow \exp m$ {piecewise exp}
9: **else**
10: $m' \leftarrow m$
11: **end if**
12: **return** m'

A.7. Additional Routines for max-out and max-out^{PP}
Algorithm 10 dominating-poly(q_1, q_2, i_1, i_2)

input q_1, q_2 : monotone polynomial or constant map to $-\infty$, i_1 lower bound interval, i_2 upper bound interval (q_1 and q_2 do not intersect on (i_1, i_2) or are equal)

output polynomial $\hat{q} \in \{q_1, q_2\}$ such that $q_1 \leq \hat{q}$ and $q_2 \leq \hat{q}$ point-wise on $[i_1, i_2]$

- 1: $q'_1 \leftarrow \text{extract-poly-piece}(q_1, i_1, i_2)$ if $(i_1, i_2) \in \text{dom}(q_1)$ else $(y \mapsto -\infty)$
- 2: $q'_2 \leftarrow \text{extract-poly-piece}(q_2, i_1, i_2)$ if $(i_1, i_2) \in \text{dom}(q_2)$ else $(y \mapsto -\infty)$
- 3: $T \leftarrow \{i \mid i \in \{i_1, i_2\}\}$
- 4: $T \leftarrow \emptyset$ if $T = \emptyset$
- 5: $e_1 \leftarrow \max\{q'_1(i) \mid i \in T\}$
- 6: $e_2 \leftarrow \max\{q'_2(i) \mid i \in T\}$
- 7: **if** $e_1 > e_2$ **then**
- 8: **return** q_2
- 9: **else if** $e_1 > e_2$ **then**
- 10: **return** q_2
- 11: **else if** $e_1 = -\infty$ **then**
- 12: **return** q'_1 {does not matter}
- 13: **else**
- 14: {We have to check what happens inside the interval, both are defined but we have the same values for our points}
- 15: $i_{\min} \leftarrow \min T$
- 16: $r \leftarrow q'_1 - q'_2$
- 17: $d \leftarrow \text{highest-non-vanishing-derivative-at}(r, i_{\min})$
- 18: **if** $d \geq 0$ **then**
- 19: {either q'_1 dominates or equal}
- 20: **return** q'_1
- 21: **else**
- 22: **return** q'_2
- 23: **end if**
- 24: **end if**

Algorithm 11 $\text{prepare-breaks}(q, l, u)$

input q : piecewise polynomial, l : affine function, u : affine function

output $V_q, C_{\hat{e}}, C_{\text{switch}}, C_{\text{bounds}}$ sets of points

- 1: $E_q \leftarrow \text{extreme-points}(q)$
 - 2: $B_q \leftarrow \text{breakpoints}(q)$
 - 3: $q'(x) \leftarrow \max(\{\lim_{x \rightarrow b^-} q(x) \text{ if defined,}$
 $\lim_{x \rightarrow b^+} q(x) \text{ if defined}\})$
 - 4: $V_q \leftarrow \{b \mapsto q'(b) \mid b \in E_q \cup B_q\}$
 - 5: $\Omega \leftarrow \{y \in \mathbb{R} \mid l(y) \leq u(y)\}$
 - 6: $C_{\hat{e}} \leftarrow \bigcup_{f \in l, u} \{f^{-1}(y) \mid y \in E_q \cup B_q \wedge f \neq \text{const.}\} \cap \Omega$
 - 7: $C_{\text{switch}} \leftarrow \text{roots}((q \circ u) - (q \circ l)) \cap \Omega$
 - 8: $C_{\text{bounds}} \leftarrow \text{bounds}(l, u)$
 - 9: **return** $V_q, C_{\hat{e}}, C_{\text{switch}}, C_{\text{bounds}}$
-

Algorithm 12 $\text{inner-max}(l, u, i_1, i_2, V_q)$

input l : affine function, u : affine function, i_1 : lower bound interval, i_2 : upper bound interval, V_q : map from point to value

output $\hat{e}$: the maximum of V_q inside $l([i_1, i_2])$ and $u([i_1, i_2])$ or $-\infty$

- 1: $l', u' \leftarrow \max\{l(i_1), l(i_2)\}, \min\{u(i_1), u(i_2)\}$
 - 2: $\hat{e} \leftarrow \max(\{V_q(b) \mid b \in V_q \wedge l' \leq b \leq u'\}$
 $\cup \{-\infty\})$
 - 3: **return** $\hat{e}$
-

Algorithm 13 $\text{simplify}(m)$

input m : piecewise polynomial

output piecewise polynomial but with redundant pieces removed

- 1: **if** $\text{is-empty}(m)$ **then**
 - 2: **return** m
 - 3: **else**
 - 4: $m' \leftarrow \text{empty-piecewise}()$
 - 5: $\$, l_{\text{last}}, u_{\text{last}}, \$'', p_{\text{last}} \leftarrow \text{to-pieces}(m)[0]$
 - 6: $\{ \$', \$'' \}$ denote whether the interval starts open/closed
 - 7: **for** $\$, l, u, \$^{end}, p \in \leftarrow \text{to-pieces}(m)[1:]$ **do**
 - 8: **if** $p = p_{\text{last}}$ **then**
 - 9: $u_{\text{last}} \leftarrow u$
 - 10: $\$'' \leftarrow \end
 - 11: **else**
 - 12: $m'|_{\$, l_{\text{last}}, u_{\text{last}} \$''} \leftarrow p'$
 - 13: $\$, l_{\text{last}}, u_{\text{last}}, \$'', p_{\text{last}} \leftarrow \$^{start}, l, u, \$^{end}, p$
 - 14: **end if**
 - 15: **end for**
 - 16: $m'|_{\$, l_{\text{last}}, u_{\text{last}} \$''} \leftarrow p'$
 - 17: **return** m'
 - 18: **end if**
-

B. Particle, Constraint-Aware Adam Optimizer

In this section, we provide details on PCADAM, the particle-based, constraints-aware version of ADAM that we present as a side contribution. We use PCADAM both as a baseline to compare with, as well as a convex-polytope optimizer combined with PAMAP in the data imputation experiments (Sec. 6). Given the constraints Δ and a density p to be maximized, and an initial batch of points $\mathcal{X}_0 = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$, PCADAM keeps track of the best feasible solution found so far, initialized to $-\infty$ (lines 1–2). Then, for a fixed number of iterations, it updates the batch of points using a gradient-based optimizer (line 5). After each update, it checks which of the new points satisfy the constraints (line 7) and updates the best solution found so far accordingly (lines 8–9). Finally, it returns the best solution found (line 13).

Algorithm 14 PCADAM($\Delta, p, it, \mathcal{X}_0$)

input Δ : SMT($\mathcal{LRA}$) formula, p : density function, $\mathcal{X}_0$: initial batch of points, it : iterations

output $\mathbf{x}^*$: global best point, m^* : maximum density found

```

1:  $m^* \leftarrow -\infty$ 
2:  $\mathbf{x}^* \leftarrow \emptyset$ 
3:  $\mathcal{X} \leftarrow \mathcal{X}_0$ 
4: for  $i = 1$  to  $it$  do
5:    $\mathcal{X} \leftarrow \text{update}(\mathbf{X}, \nabla p)$  {Gradient-based step}
6:   for each  $\mathbf{x} \in \mathcal{X}$  do
7:     if  $\mathbf{x} \models \Delta$  and  $p(\mathbf{x}) > m^*$  then
8:        $m^* \leftarrow p(\mathbf{x})$ 
9:        $\mathbf{x}^* \leftarrow \mathbf{x}$ 
10:    end if
11:  end for
12: end for
13: return  $\mathbf{x}^*, m^*$ 

```

C. Experiments

C.1. Implementation

We have implemented our algorithms in Python, building on top of several existing libraries. For PCADAM, we have used Pytorch’s Adam optimizer for parallel unconstrained optimization. For PAMAP, we have used the SAE4WMI enumeration algorithm (Spallitta et al., 2024) implemented in wmpy³; for numerical constrained optimization over convex polytopes, we have used SciPy’s optimization routines; for Lasserre’s method, we implemented the moment hierarchy on top of SumOfSquares.py.⁴ For MP-MAP, we have implemented our message-passing using SymPy for the symbolic computations over the polynomials and pySMT in order to query SMT solvers for logical operations. Parts of the code are adapted from MP-WMI (Zeng et al., 2020a), which has a similar high-level structure.

C.2. Details for STAR, SNOW and PATH

We start by sampling a random, N -variable SMT formula of the shape of either *STAR* (star-shaped primal graph), *SNOW* (ternary-tree shaped primal graph) or *PATH* (linear-chain shaped primal graph). On the resulting support, we generate a non-negative, piecewise polynomial function by first randomly selecting literals to attach the polynomials to. Our weight-functions then looks like this: $\prod_{l \in \text{random-literals}} (\text{if } \mathbf{x} \models l \text{ then } q_l(\mathbf{x}) \text{ else } 1)$. As each literal can at most mention two variables, the polynomial can also only be over those two variables, as it has to respect the scope of the literal it is attached to. Furthermore, as we want to have separable polynomials, we generate the polynomials by generating two univariate polynomials of degree deg and form the product. Each univariate polynomial is generated as follows:

Algorithm 15 `random-poly-univar`($var, deg, bound_{lower}, bound_{upper}, pareto_{upper}$)

```

1:  $generateDegree \leftarrow \text{int}(deg/2)$ 
2:  $numRoots \leftarrow generateDegree - 1$ 
3:  $roots \leftarrow [r \sim U[bound_{lower}, bound_{upper}] \mid r \in [1, numRoots]]$ 
4:  $sample\_coeff \leftarrow \lambda$ . return  $s \sim \text{TruncatedPareto}(min = 2.0, max = pareto_{upper}, eps = 0.01)$ 
5:  $mons \leftarrow [(sample\_coeff() \cdot (var - r)) \mid r \in roots]$ 
6:  $poly_{derivative}^{unsquared} \leftarrow \prod_{m \in mons} m$ 
7:  $poly^{unsquared} \leftarrow \text{integrate}(poly_{derivative}^{unsquared}, var)$ 
8: return  $(poly^{unsquared}(var))^2 + 1$ 

```

Additionally, for the literals specifying the global bounds (so $x_i \geq bound_{lower}$ and $x_i \leq bound_{upper}$), we attach the polynomial $q_{global}(x_i) = (x - bound_{lower}) * (bound_{upper} - x)$, as otherwise the maximum is too commonly found at the global bounds.

We generate the inequalities by sampling two random points inside our global bounds, computing the connecting line which forms our decision boundary, and randomly choosing either the left or right as the valid half-space.

We generate the datasets with the following configurations. For each shape $s \in \{\text{STAR}, \text{SNOW}, \text{PATH}\}$ we generate problems over the combination of the following parameters:

- N : 2, 4, 6, 8, 10
- deg (squared, per univariate poly): 2, 3, 4
- $n_{clauses}$: 2, 3
- $n_{literals}$: 2, 4
- $pareto_{upper}$: $N \leq 6 \Rightarrow 15, N \geq 8 \Rightarrow 2.5$,
- $bound_{lower}$: -1
- $bound_{upper}$: 1

³<https://github.com/unitn-sml/wmpy>

⁴<https://github.com/yuanchenyang/SumOfSquares.py>

Of each configuration, we generate 2 random problems.

Here, the number of random literals $n_{literals}$ is per clause (so times $n_{clauses}$). Additionally, we have the inequalities specifying the global bounds.

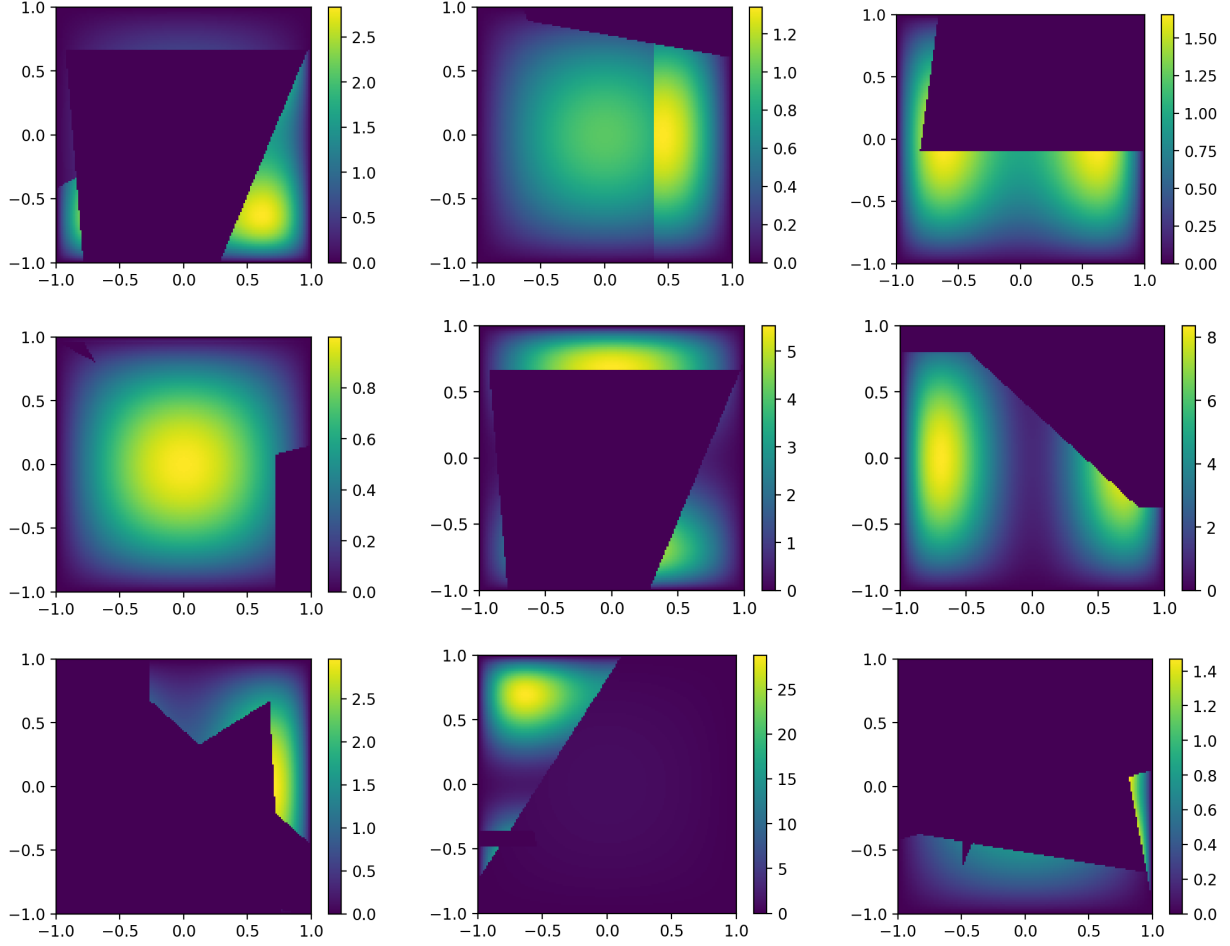


Figure 9. Random examples of the resulting density for problems of shape *PATH* in the 2d-case.

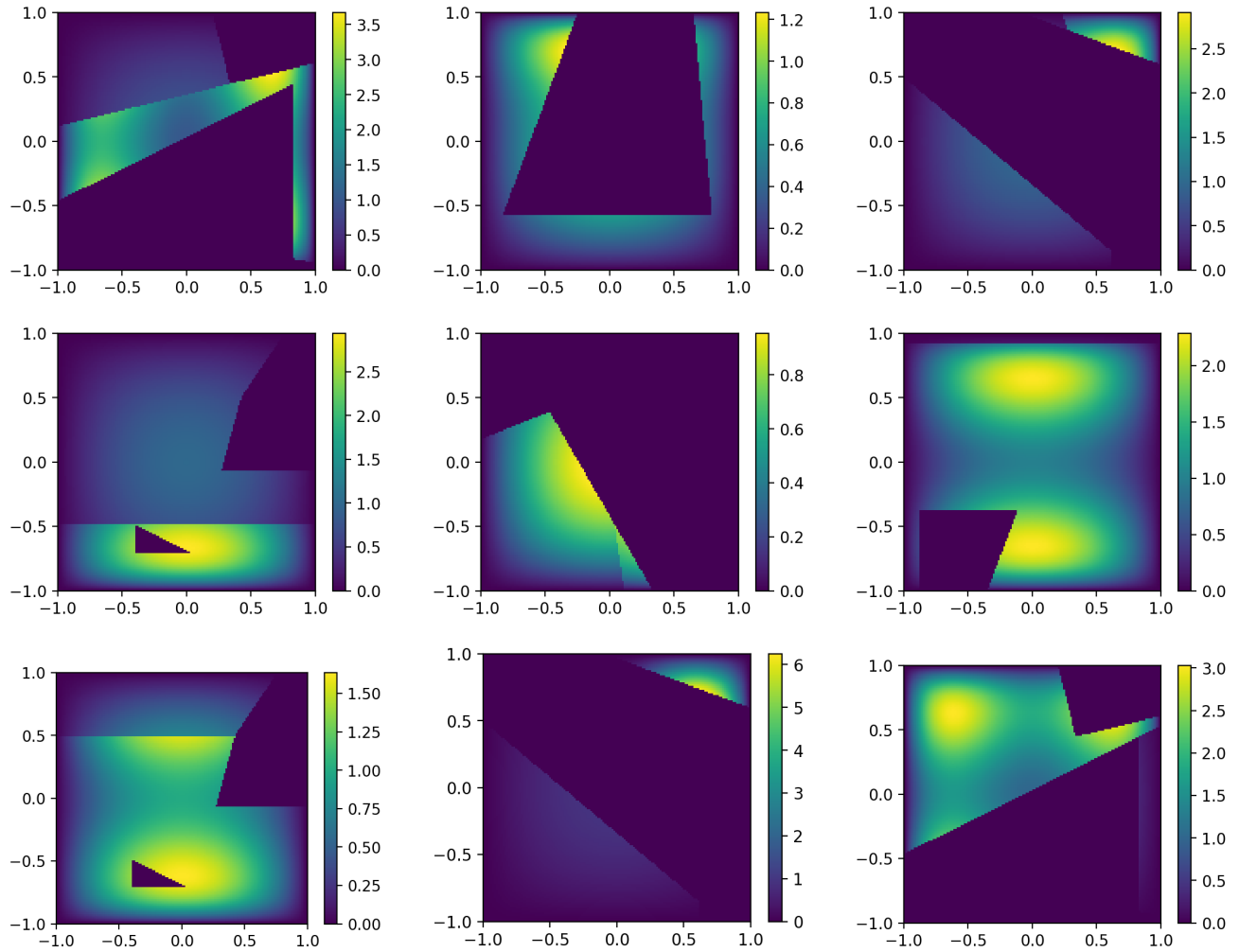


Figure 10. Random examples of the resulting density for problems of shape *SNOW* in the 2d-case.

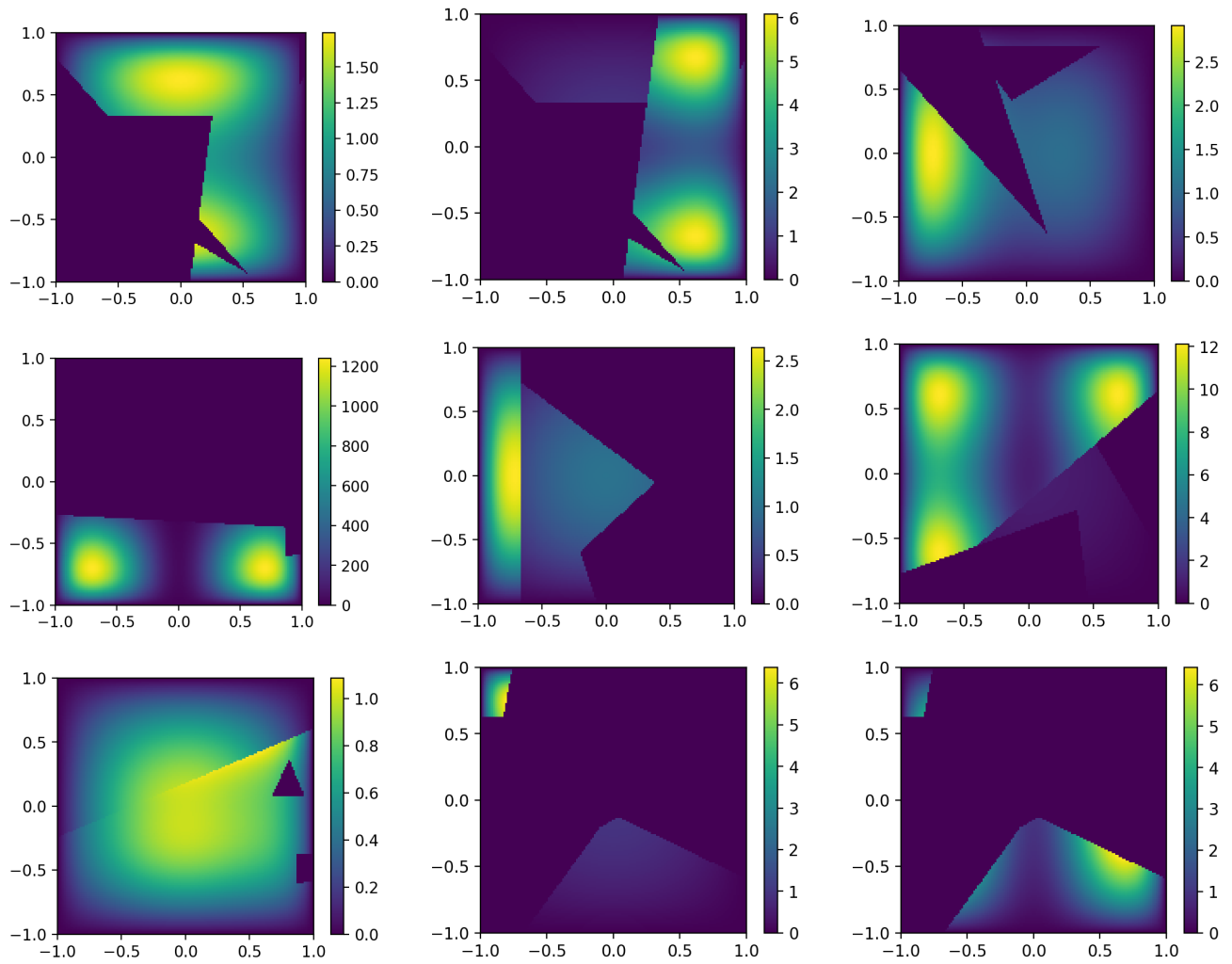


Figure 11. Random examples of the resulting density for problems of shape *STAR* in the 2d-case.

C.3. Hyperparameters for the optimizers for STAR, SNOW and PATH

In general, the optimizers we compare the MP-MAP to are run with increasing budget until either we have a time-out or we have a relative error of 0.01 compared to the grid-search baseline.

PA-SHGO We run PA-SHGO with the following hyperparameters:

- **local-optimizers** cobyla
- **budget (iters)** 1 2 4 16 32 64 128 256

ParticleAdam We run ParticleAdam with the following hyperparameters:

- **lr** 0.001
- **max-iterations** 2500
- **budget (particles)** 100 1000 10000 100000 1000000

Baseline Recursive Grid-Search We run the baseline (grid search with a recursion into the best found point) with the following hyperparameters:

- **grid size (per dim)** 10
- **precision** double
- **number of recursions** 30
- **recursion shrink factor** 0.2 (shrinks the domain in the recursion)

This grid search is run per polytope enumerated, via the domain of the bounding box of the polytope. It is therefore used as a local optimizer in Alg. 4. It constitutes a simple global search baseline that is intentionally easy to outperform. As MPMAP returns the global optimum, it always beats this baseline and is therefore always accepted.

C.4. Results on STAR, SNOW, PATH

Num Variables	MP-MAP	PA-SHGO	ParticleAdam
2	0.05 $\pm$ 0.03	0.01 $\pm$ 0.00	5.03 $\pm$ 9.44
4	0.14 $\pm$ 0.12	0.33 $\pm$ 0.36	11.90 $\pm$ 12.80
6	1.77 $\pm$ 6.05	5.07 $\pm$ 9.57	22.74 $\pm$ 11.49
8	3.85 $\pm$ 8.25	15.51 $\pm$ 14.51	18.79 $\pm$ 12.20
10	8.46 $\pm$ 12.48	23.27 $\pm$ 11.40	23.28 $\pm$ 10.47
12	5.25 $\pm$ 9.55	30.00 $\pm$ 0.00	23.45 $\pm$ 9.72
14	7.05 $\pm$ 10.27	-	-
16	6.56 $\pm$ 10.32	-	-
18	12.01 $\pm$ 12.84	-	-
20	10.46 $\pm$ 12.87	-	-
22	17.01 $\pm$ 14.14	-	-
24	13.34 $\pm$ 13.01	-	-
26	18.90 $\pm$ 13.25	-	-

Table 2. Runtime comparison on the SNOW-dataset in mean and standard deviations for our approaches.

Num Variables	MP-MAP	PA-SHGO	ParticleAdam
2	0.07 $\pm$ 0.06	0.02 $\pm$ 0.01	3.85 $\pm$ 7.88
4	1.39 $\pm$ 5.97	0.29 $\pm$ 0.30	10.83 $\pm$ 12.31
6	1.53 $\pm$ 6.07	2.83 $\pm$ 3.50	17.19 $\pm$ 12.82
8	1.70 $\pm$ 5.92	10.34 $\pm$ 11.15	22.50 $\pm$ 11.35
10	1.83 $\pm$ 5.90	19.59 $\pm$ 11.39	24.26 $\pm$ 9.95
12	2.01 $\pm$ 6.13	25.70 $\pm$ 9.35	23.29 $\pm$ 10.16
14	0.84 $\pm$ 0.57	-	-
16	7.56 $\pm$ 12.20	-	-
18	3.56 $\pm$ 8.20	-	-
20	2.68 $\pm$ 6.21	-	-
22	11.08 $\pm$ 13.48	-	-
24	10.05 $\pm$ 12.93	-	-
26	13.69 $\pm$ 13.85	-	-

Table 3. Runtime comparison on the STAR-dataset in mean and standard deviations for our approaches.

Num Variables	MP-MAP	PA-SHGO	ParticleAdam
2	0.04 $\pm$ 0.02	0.02 $\pm$ 0.00	9.79 $\pm$ 12.97
4	1.08 $\pm$ 2.22	0.19 $\pm$ 0.16	14.75 $\pm$ 13.37
6	6.56 $\pm$ 9.54	8.18 $\pm$ 10.70	19.04 $\pm$ 12.66
8	11.54 $\pm$ 12.55	18.81 $\pm$ 11.81	17.78 $\pm$ 12.23
10	12.08 $\pm$ 12.50	21.57 $\pm$ 11.62	23.40 $\pm$ 10.28
12	14.55 $\pm$ 12.91	30.00 $\pm$ 0.00	21.07 $\pm$ 10.58
14	17.97 $\pm$ 13.14	-	-
16	12.51 $\pm$ 12.55	-	-
18	20.05 $\pm$ 12.93	-	-
20	20.89 $\pm$ 11.91	-	-
22	18.78 $\pm$ 11.85	-	-
24	22.71 $\pm$ 10.17	-	-
26	24.38 $\pm$ 8.60	-	-

Table 4. Runtime comparison on the PATH-dataset in mean and standard deviations for our approaches.

C.5. Details for SDD experiments

We provide additional experiments on the SDD dataset, and on the configuration of the optimization algorithms used in our experiments. We recall that for this dataset, we learn the densities with PAL (Kurscheidt et al., 2025). PAL densities are non-negative piecewise polynomials, each piece being an axis-aligned box $\Phi = [l_1, u_1] \times \cdots \times [l_N, u_N] \subseteq \mathbb{R}^N$ over which the density is defined as

$$p_\Phi(\mathbf{x}) = \sum_{i=1}^d \alpha_i \prod_{j=1}^N s_j(x_j)^2, \quad (70)$$

where each $s_j(x_j)$ is a cubic polynomial. As such, they can be seen as simple squared probabilistic circuits (PCs) (Choi et al., 2020; Vergari et al., 2021) which have been recently investigated in the PC literature for their expressiveness properties (Loconte et al., 2024; 2025b;a).

C.5.1. PCADAM

In order to better understand the performance PCADAM, we conduct experiments over different hyperparameter configurations. We fix the learning rate to 0.1 and vary:

- N : number of parallel particles used in the optimization;
- it : number of iterations for each particle;

The results are shown in Fig. 12. As expected, all configurations report feasible solutions, but the performance varies significantly. Since the SDD dataset is low-dimensional (2D) and the feasible region is broad, PCADAM can find good solutions if enough particles and iterations are used. However, this comes at the cost of increased computation time.

C.5.2. PAMAP

Enumeration and upper bounds. For these densities, we implement the enumerator as follows. First, we compute the upper bound for each spline piece (see below). Then we sort the pieces in decreasing order of their upper bounds, so as to increase the chances of pruning suboptimal pieces early. Finally, we enumerate polytopes corresponding to each piece in turn, terminating when the upper bound of the current piece is lower than the current best value.

An upper bound of the maximum of (70) over a piece Φ can be computed as follows:

$$\max_{\mathbf{x} \in \Phi} p_\Phi(\mathbf{x}) \leq \sum_{i=1}^d \alpha_i \cdot \max_{\mathbf{x} \in \Phi} \prod_{j=1}^N s_j(x_j)^2 \quad (71)$$

$$= \sum_{i=1}^d \alpha_i \prod_{j=1}^N \max_{x_j \in [l_j, u_j]} s_j(x_j)^2 \quad (72)$$

The critical points of $s_j(x_j)^2$ are the same as those of $s_j(x_j)$, which can be computed in closed form. Thus, we evaluate $s_j(x_j)$ at its critical points within $[l_j, u_j]$, as well as at the interval boundaries l_j and u_j . The maximum squared value among these points provides the desired bound. Fig. 13 shows how pruning works in practice on 10 sample trajectories from the SDD dataset.

SciPy’s optimizers. For PAMAP(SciPy), we tried all the combinations of global and local constrained optimizers available in SciPy’s optimize library. The results are shown in Fig. 12b.

From the plot, we can see that the fastest configuration is the combination of SHGO as global optimizer and SLSQP as local optimizer, achieving comparable relative error as the other configurations, hence the one we used in our experiments.

C.5.3. OMT ENCODING

OMT($\mathcal{NRA}$) solvers require the optimization problem to be encoded as a pair of a logical formula and an objective function encoded as a $\mathcal{NRA}$ -term. While the logical formula is directly given by the problem, multiple choices are possible for encoding the objective function (70).

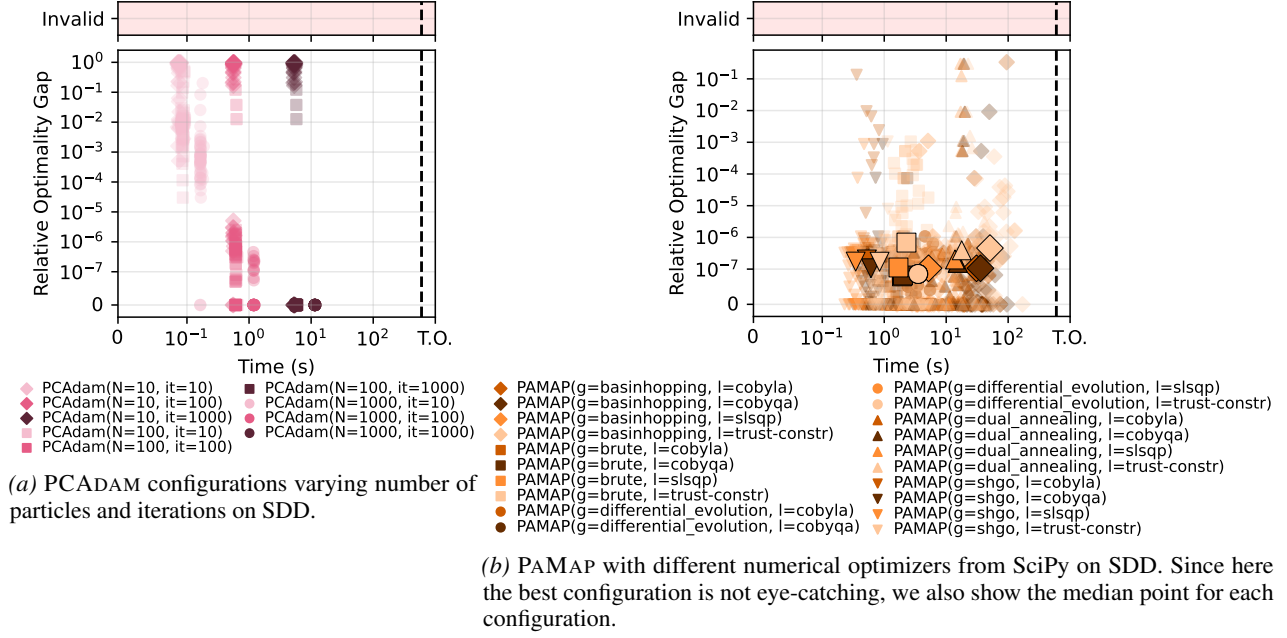


Figure 12. Experiments on SDD dataset varying PCADAM hyperparameters (left) and PAMAP numerical optimizers (right).

A first possibility is to encode it as a nested if-then-else expression:

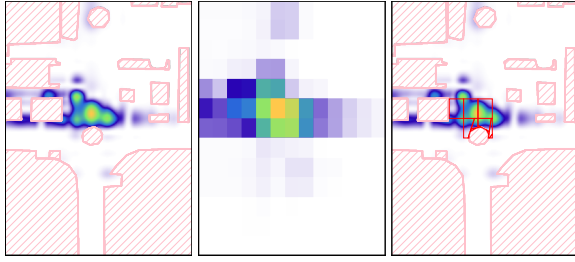
$$\text{Ite}(\Phi_1, p_{\Phi_1}(\mathbf{x}), \text{Ite}(\Phi_2, p_{\Phi_2}(\mathbf{x}), \dots)) \quad (73)$$

where each Φ_i is a piece of the PAL density. An alternative is to encode the objective as a sum of if-then-else expressions:

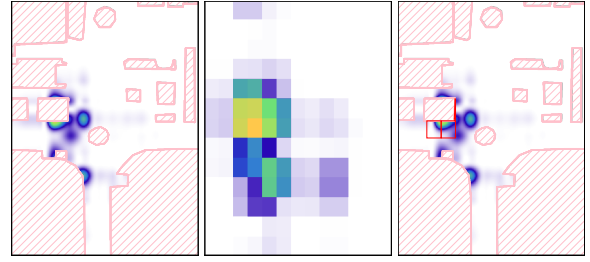
$$\sum_{\Phi_i} \text{Ite}(\Phi_i, p_{\Phi_i}(\mathbf{x}), 0). \quad (74)$$

To ensure that this encoding is well defined, the regions Φ_i must be mutually exclusive. We enforce this by defining each region using left-closed, right-open intervals, except for the final region along each dimension, which is closed on the right as well.

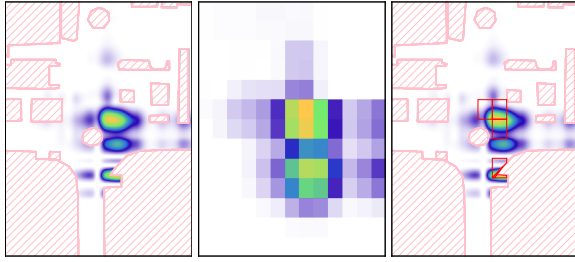
We tested both encodings using OptiMathSAT and CDCL-OCAC. The first encoding was challenging for both OMT solvers, and both timed out without finding a solution. With the second encoding, OptiMathSAT in anytime mode occasionally found a solution within the time limit, exiting with an error the remaining times. In contrast, CDCL-OCAC timed out on both encodings without finding any solution.



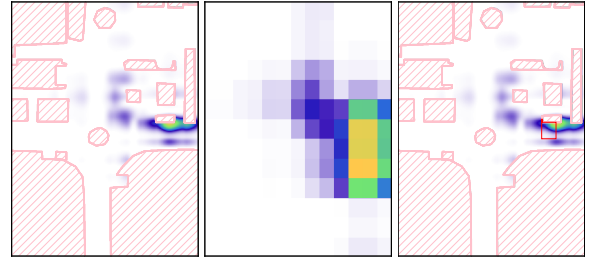
Traj. 2000. Enumerated polytopes: 12/257



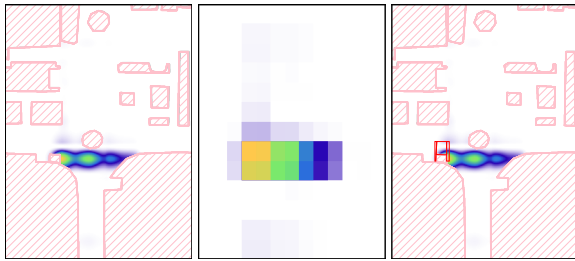
Traj. 3200. Enumerated polytopes: 4/257



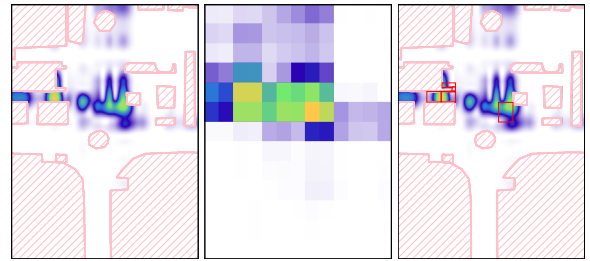
Traj. 6000. Enumerated polytopes: 6/257



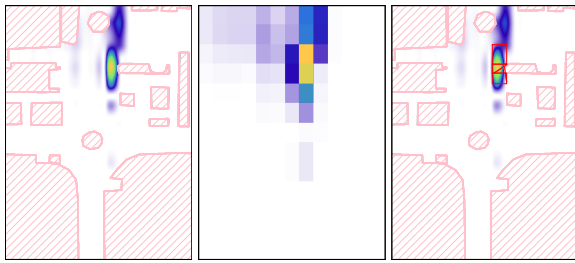
Traj. 9000. Enumerated polytopes: 2/257



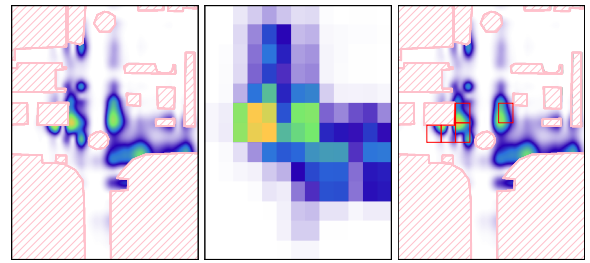
Traj. 13000. Enumerated polytopes: 3/257



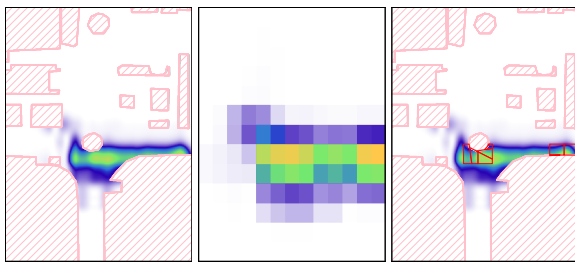
Traj. 17000. Enumerated polytopes: 9/257



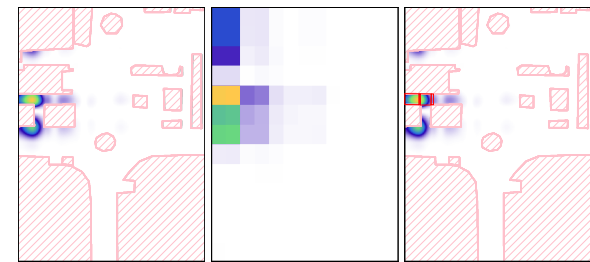
Traj. 22000. Enumerated polytopes: 6/257



Traj. 29300. Enumerated polytopes: 7/257



Traj. 35200. Enumerated polytopes: 10/257



Traj. 41600. Enumerated polytopes: 4/257

Figure 13. Visualization of the PAMAP pruning process across 10 sample trajectories. Each group shows the predictive density (left), the computed upper bounds used for pruning (center), and the polytopes analyzed by PAMAP(SHGO) thanks to pruning (right).

C.6. Details For Constrained MAP-prediction For Imputation On Tabular Data

C.6.1. THE TVAE OPTIMIZATION OBJECTIVE

In order to train a TVAE (Xu et al., 2019), our optimization objective is the common ELBO-style optimization objective extended to handle categorical data:

$$\text{loss-item}(x_i, \hat{x}_i, \hat{\sigma}_i) = \begin{cases} \frac{1}{2\sigma_i^2}(x_i - \hat{x}_i)^2 - \log \sigma_i, & \text{if } \neg(\text{is-categorical}(x_i)) \\ \text{CrossEntropy}(x_i, \hat{x}_i) & \end{cases} \quad (75)$$

$$\mathcal{L}_{\text{rec}}(\mathbf{x}, \hat{\mathbf{x}}, \hat{\sigma}) = \frac{1}{N} \sum_{b=1}^N \sum_i \text{loss-item}(x_i^b, \hat{x}_i^b, \hat{\sigma}_i^b) \quad (76)$$

$$\mathcal{L}_{KL}(\boldsymbol{\mu}_{\text{latent}}, \boldsymbol{\sigma}_{\text{latent}}) = \frac{1}{N} \left(\sum_{b=1}^N -\frac{1}{2} \sum_i (1 + \log(((\sigma_{\text{latent}})_i^b)^2) - ((\mu_{\text{latent}})_i^b)^2 - ((\sigma_{\text{latent}})_i^b)^2) \right) \quad (77)$$

$$\mathcal{L}(\mathbf{x}) = \mathcal{L}_{\text{rec}}(\mathbf{x}, \hat{\mathbf{x}}) + \mathcal{L}_{KL}(\boldsymbol{\mu}_{\text{latent}}, \boldsymbol{\sigma}_{\text{latent}}) \quad (78)$$

$$\text{where } (\boldsymbol{\mu}_{\text{latent}}, \boldsymbol{\sigma}_{\text{latent}}) = \text{encoder}_{NN}(\mathbf{x}) \quad (79)$$

$$\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}_{\text{latent}}, \boldsymbol{\sigma}_{\text{latent}}) \quad (80)$$

$$(\hat{\mathbf{x}}, \hat{\sigma}) = \text{decoder}_{NN}(\mathbf{z}) \quad (81)$$

In order to compute $cMAP(p, \Delta) = \arg\max_{\mathbf{x} \in \Delta} p(\mathbf{x})$, we have to decide for a p to optimize. We essentially have two options: We can directly optimize the ELBO-relaxation of p that is detailed above, which is a stochastic objective, or we can optimize over $\mathbf{x}$ and $\mathbf{z}$ jointly and then discard $\mathbf{z}$. The second objective is a common starting point to escape the stochastic nature of the first (González et al., 2022).

C.6.2. DISCRETE VARIABLES

Similar to Stoian & Giunchiglia (2025), we treat the discrete variables as continuous from the point of view of our model.

C.6.3. RESULTS

We benchmark our methods on the *House-Price* prediction dataset with the constraints provided by Stoian & Giunchiglia (2025). We train a TVAE-model according to the hyperparameters provided by Stoian & Giunchiglia (2025), so with 150 epochs, batch size 70, l2-scale 0.0002, learning rate 0.0002 (we use Adam) and loss-factor 2.

In order to perform our MAP-prediction, we use 100 samples from the latent in order to estimate $p(x_m)$, and use a learning rate of 0.1. We provide detailed results, including the seconds per sample, in table 5.

In order to generate the starting points, for the unconstrained baselines, we sample from the model, and for PAMAP(PCADAM) we first sample unconstrained and then project into the current enumerated polytope.

For DL2, we use a λ of 0.6, but provide additional results for other values of λ in table 5.

Method	Particles	Mean RE	Median RE	Trimmed Mean RE	Sec./Sample
PAMAP(PCADAM)	10	0.2270	0.0011	0.1985	4.38
Adam - unconstrained	10	33.1228	0.2216	16.7370	3.23
Adam - unconstrained	100	25.8524	0.2244	10.5278	3.46
DL2($\lambda = 0.1$)	10	15.6419	0.0893	4.3880	3.08
DL2($\lambda = 0.1$)	100	12.4286	0.0836	1.7082	3.02
DL2($\lambda = 0.6$)	10	12.2241	0.0597	2.1966	3.10
DL2($\lambda = 0.6$)	100	16.7852	0.0308	1.5258	3.05
DL2($\lambda = 1.0$)	10	13.2146	0.0561	2.9426	3.04
DL2($\lambda = 1.0$)	100	17.0413	0.0262	1.7844	2.96

Table 5. Imputation error statistics (aggregated over the dataset). We provide the average relative l2-distance, denoted by Mean RE, the Median RE and the Trimmed Mean RE (Right, 1%) over the methods. Additionally, the timings are given in seconds per sample.